

Quantitative Data Analysis in Finance

Liam O'Shaughnessy

April 24, 2026

Contents

1	Introduction	2
2	Regression	5
3	Model Selection	17
4	Factor Models	22
5	Ensemble Methods	26
6	Tree-Based Methods	28
7	Sieves	34
8	Clustering	38
9	Classification	42
10	Neural Networks	50
11	Large Language Models	55

Preface

These are my notes from when I took ECO 480/FIN 580 in Spring 2026 at Princeton University. I have elaborated considerably on the main content with supplementary material, proofs, and examples.

1 Introduction

Can machines learn finance? Artificial intelligence (AI) has exploded in success, with areas such as ML and NLP. Successes with DeepBlue, Watson, real-time translation, AlphaGo, AlphaFold, and the ensemble of LLMs have shown great promise. “Statistical AI” has relied on giving computers subproblems and rules for solving specific problems, whereas “Neural AI” has relied on complex, often unexplainable architectures equipped with big data to solve problems. The main drivers of these have been hardware (CPUs, GPUs, TPUs), software (architectures), and big data (now collecting incredible volumes daily). Three subtypes of machine learning are supervised (labeled data, such as regression and classification), unsupervised (unlabeled data, such as PCA and anomaly detection), and reinforcement (trial and error to maximize rewards).

Definition 1.1 (Supervised Learning Setup). Let $\{(\vec{x}_i, y_i)\}_{i=1}^n$ be a dataset of n observations, where $\vec{x}_i \in \mathbb{R}^p$ is a vector of p features (covariates) and $y_i \in \mathbb{R}$ is a scalar response. Collect observations into a data matrix $X \in \mathbb{R}^{n \times p}$ and response vector $\vec{y} \in \mathbb{R}^n$. The goal of *supervised learning* is to find a function $\hat{f} : \mathbb{R}^p \rightarrow \mathbb{R}$ that predicts y well for new, unseen inputs $\vec{x}$, by minimizing some loss function L over the training data.

Definition 1.2 (Return). The *simple return* of an asset with price P_t over one period, the dividend payment of which is D_t , is

$$R_t = \frac{P_t + D_t - P_{t-1}}{P_{t-1}} = \frac{X_t - P_{t-1}}{P_{t-1}}. \quad (1)$$

Definition 1.3 (Euler Equation). The canonical asset pricing formula in discrete time periods, as is reproducible from analogous martingale pricing, is

$$P_t = \mathbb{E}[M_{t+1}X_{t+1}|\mathcal{F}_t]. \quad (2)$$

Definition 1.4 (Stochastic Discount Factor). A *stochastic discount factor* is any function M_t for which the Euler equation holds for an asset. For a utility function u of consumption, the CCAPM proposes a consumption-based SDF as a function of the consumer’s subjective discount factor β :

$$M_{t+s} = \frac{\beta u'(c_{t+s})}{u'(c_t)}. \quad (3)$$

Theorem 1.1 (Asset Pricing). *Given a risk-free asset with return $R_{f,t}$, the return on the i th asset*

$R_{i,t}$ satisfies the following:

$$\mathbb{E}[R_{i,t+1}|\mathcal{F}_t] - R_{f,t} = \frac{\text{Cov}_t(M_{t+1}, R_{i,t+1})}{\text{Var}_t(M_{t+1})} (-R_{f,t} \text{Var}_t(M_{t+1})) \quad (4)$$

$$= \beta_{i,t} \lambda_t. \quad (5)$$

λ_t is referred to as the “price of risk”.

Proof. Consider the Euler equation for asset pricing. Then

$$P_{i,t} = \mathbb{E}[M_{t+1} X_{i,t+1} | \mathcal{F}_t] \quad (6)$$

$$= \mathbb{E}[M_{t+1} R_{i,t+1} P_{i,t} | \mathcal{F}_t], \quad (7)$$

so $1 = \mathbb{E}[M_{t+1} R_{i,t+1} | \mathcal{F}_t]$, the price is $\mathcal{F}_t$ -measurable. Additionally, for a risk-free asset paying 1 at time $t + 1$, under no-arbitrage,

$$P_{f,t} = \frac{1}{R_{f,t}} = \mathbb{E}[M_{t+1} 1 | \mathcal{F}_t] \quad (8)$$

$$R_{f,t} = \frac{1}{\mathbb{E}[M_{t+1} | \mathcal{F}_t]}, \quad (9)$$

since the return is $\mathcal{F}_t$ -measurable. Rewriting expectation of a product as

$$1 = \mathbb{E}[M_{t+1} | \mathcal{F}_t] \mathbb{E}[R_{i,t} | \mathcal{F}_t] + \text{Cov}_t(M_{t+1}, R_{i,t+1}), \quad (10)$$

substituting the SDF gives

$$R_{f,t} = \mathbb{E}[R_{i,t} | \mathcal{F}_t] + R_{f,t} \text{Cov}_t(M_{t+1}, R_{i,t+1}). \quad (11)$$

Rearranging this gives the theorem. □

Example 1.1 (CAPM as a Single-Factor Model). The Capital Asset Pricing Model (CAPM) is the simplest instance of the Asset Pricing Theorem. Let the SDF be linear in the market return $R_{m,t+1}$. Then the expected excess return on any asset i is

$$\mathbb{E}[R_{i,t+1}] - R_f = \beta_i (\mathbb{E}[R_{m,t+1}] - R_f), \quad \beta_i = \frac{\text{Cov}(R_i, R_m)}{\text{Var}(R_m)}. \quad (12)$$

Suppose the market excess return is $\mathbb{E}[R_m] - R_f = 6\%$ per year. A stock with $\beta_i = 1.3$ has an expected excess return of $1.3 \times 6\% = 7.8\%$. A defensive utility stock with $\beta_i = 0.5$ has an expected excess return of only 3%. Empirically, “high-beta” stocks have historically underperformed their CAPM-predicted returns (the “betting-against-beta” anomaly).

Remark 1.1 (Factor Models in Practice). The Fama–French three-factor model extends the CAPM by adding size (SMB: Small Minus Big) and value (HML: High Minus Low book-to-market) factors.

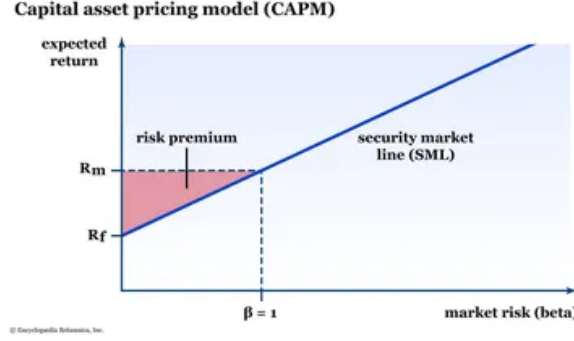


Figure 1: CAPM

A time-series regression of excess returns on these factors,

$$R_{i,t} - R_{f,t} = \alpha_i + \beta_{i,1}(R_{m,t} - R_{f,t}) + \beta_{i,2}\text{SMB}_t + \beta_{i,3}\text{HML}_t + \varepsilon_{i,t}, \quad (13)$$

tests whether $\alpha_i = 0$ (the asset is correctly priced by the model). A statistically significant α_i indicates abnormal return unexplained by the factors.

Theorem 1.2 (Bias–Variance). *For $\hat{f}(x)$ a function predicting $\{y\}$ observed data, generated by an underlying process, the mean squared error decomposes as*

$$\text{MSE} = \text{Bias}[\hat{f}(x)]^2 + \text{Var}[\hat{f}(x)] + \sigma^2, \quad (14)$$

establishing a fundamental trade-off: reducing bias (by fitting a more complex model) tends to increase variance, and vice versa. Regularization methods navigate this trade-off by deliberately introducing bias to achieve a larger reduction in variance.

Proof. Consider the observed data as generated by some underlying process with Gaussian errors $f(x) + \varepsilon$, then the error is

$$\text{MSE} = \mathbb{E}[(y - \hat{f}(x))^2] = \mathbb{E}[(\varepsilon + f(x) - \hat{f}(x))^2] \quad (15)$$

$$= \mathbb{E}[(f(x) - \hat{f}(x))^2] + 2\mathbb{E}[(f(x) - \hat{f}(x))\varepsilon] + \mathbb{E}[\varepsilon^2] \quad (16)$$

$$= \mathbb{E}[(f(x) - \mathbb{E}[\hat{f}(x)] + \mathbb{E}[\hat{f}(x)] - \hat{f}(x))^2] + 0 + \sigma^2. \quad (17)$$

Expanding this and simplifying yields

$$\text{MSE} = (f(x) - \mathbb{E}[\hat{f}(x)])^2 + \mathbb{E}[(\mathbb{E}[\hat{f}(x)] - \hat{f}(x))^2] + \sigma^2 \quad (18)$$

$$= \text{Bias}[\hat{f}(x)]^2 + \text{Var}[\hat{f}(x)] + \sigma^2. \quad (19)$$

□

Example 1.2 (Bias–Variance in Return Prediction). Consider predicting next-month stock returns r_{t+1} from 50 firm characteristics. A linear model using all 50 features has low bias (can capture all linear relationships) but high variance (many parameters estimated from limited data). A model

using only 5 characteristics has higher bias (may miss relevant predictors) but much lower variance. With a typical panel of $n = 3000$ stocks and $T = 240$ months, the sparser model tends to win in out-of-sample R^2 because the variance reduction outweighs the added bias.

Definition 1.5 (Random Vectors). Let $\vec{X} \in \mathbb{R}^n$ be a *random vector* with components $\vec{X} = (X_1, \dots, X_n)^T$, each random variables. The expectation is defined componentwise:

$$\mathbb{E}[\vec{X}] = \begin{pmatrix} \mathbb{E}[X_1] \\ \vdots \\ \mathbb{E}[X_n] \end{pmatrix}. \quad (20)$$

The variance of a random vector is its covariance matrix:

$$\text{Var}(\vec{X}) = \mathbb{E}[(\vec{X} - \mathbb{E}[\vec{X}])(\vec{X} - \mathbb{E}[\vec{X}])^T]. \quad (21)$$

Equivalently,

$$\text{Var}(\vec{X}) = \mathbb{E}[\vec{X}\vec{X}^T] - \mathbb{E}[\vec{X}]\mathbb{E}[\vec{X}]^T. \quad (22)$$

Considering $\text{Var}(\vec{X})_{ij} = \text{Cov}(X_i, X_j)$, the diagonal entries are variances and the off-diagonal entries are covariances.

Proposition 1.1 (Linear Transformations). *If $\vec{Y} = B\vec{X} + \vec{c}$ for matrix B and vector $\vec{c}$, then*

$$\mathbb{E}[\vec{Y}] = B \mathbb{E}[\vec{X}] + \vec{c}, \quad (23)$$

$$\text{Var}(\vec{Y}) = B \text{Var}(\vec{X}) B^T. \quad (24)$$

Proof. Linearity of expectation gives $\mathbb{E}[\vec{Y}] = B \mathbb{E}[\vec{X}] + \vec{c}$ directly. For the variance,

$$\text{Var}(\vec{Y}) = \mathbb{E}[(\vec{Y} - \mathbb{E}[\vec{Y}])(\vec{Y} - \mathbb{E}[\vec{Y}])^T] \quad (25)$$

$$= \mathbb{E}[B(\vec{X} - \mathbb{E}[\vec{X}])(B(\vec{X} - \mathbb{E}[\vec{X}]))^T] \quad (26)$$

$$= B \mathbb{E}[(\vec{X} - \mathbb{E}[\vec{X}])(\vec{X} - \mathbb{E}[\vec{X}])^T] B^T = B \text{Var}(\vec{X}) B^T, \quad (27)$$

since the constant $\vec{c}$ cancels in the centering and B can be pulled out of the expectation. $\square$

2 Regression

Theorem 2.1 (Singular Value Decomposition). *For complex $m \times n$ matrix A , there exists a decomposition*

$$A = U \Sigma V^\dagger, \quad (28)$$

where U is an $n \times n$ unitary matrix, Σ is a rectangular diagonal $m \times n$ matrix, and V is an $m \times m$ unitary matrix.

Proof. Consider the $n \times n$ matrix $A^T A$. This matrix is symmetric and positive definite ($\vec{x}^T A^T A \vec{x} = \|A\vec{x}\|^2 \geq 0$), so it is diagonalizable (by spectral theorem) and has non-negative eigenvalues $\{\lambda_i\}$. Therefore $A^T A = V \Lambda V^T$. Define the singular values of $A^T A$ as $\sigma_i = \sqrt{\lambda_i}$, which are real, so that $A^T A \vec{v}_i = \sigma_i^2 \vec{v}_i$ for eigenvectors $\vec{v}_i$.

Call $\vec{u}_i = \frac{1}{\sigma_i} A \vec{v}_i$. It is straightforward to see that the $\vec{u}_i$ s are orthonormal; if there are a set of eigenvectors for which $A \vec{v}_i = 0$, the Gram-Schmidt process can be employed to find an orthonormal basis in the null space. Hence, $A \vec{v}_i = \sigma_i \vec{u}_i$, so $AV = U\Sigma$, where the largest singular values are put first on the diagonal. $\square$

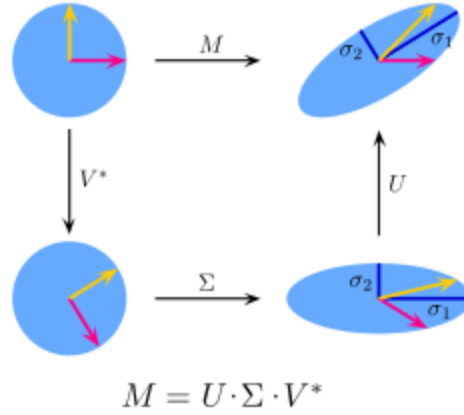


Figure 2: SVD mapping

Remark 2.1. If X is a matrix of data, the singular value decomposition can be written $X = \sum_{i=1}^{\min(m,n)} \sigma_i \vec{u}_i \vec{v}_i^T$. A rank k approximation (or “reduced SVD”) of the data can thus be written

$$X_k = \sum_{i=1}^k \sigma_i \vec{u}_i \vec{v}_i^T. \quad (29)$$

Theorem 2.2 (Eckart-Young). *The singular value decomposition is the best rank- k approximation of a matrix X — that is, for the Frobenius norm,*

$$X_k = \arg \min_{\text{rank}(Y) \leq k} \|X - Y\|_F, \quad (30)$$

$$\|X - X_k\|_F^2 = \sum_{i>k} \sigma_i^2. \quad (31)$$

Proof. Write the full SVD as $X = U \Sigma V^T = \sum_{i=1}^r \sigma_i \vec{u}_i \vec{v}_i^T$, where $r = \text{rank}(X)$. For any matrix Y with $\text{rank}(Y) \leq k$, the null space of Y has dimension at least $n - k$. Consider the $(k+1)$ -dimensional subspace $W = \text{span}(\vec{v}_1, \dots, \vec{v}_{k+1})$. By a dimension argument, $W \cap \ker(Y) \neq \{0\}$, so there exists a

unit vector $\vec{w} \in W$ with $Y\vec{w} = 0$. Writing $\vec{w} = \sum_{i=1}^{k+1} c_i \vec{v}_i$ with $\sum c_i^2 = 1$,

$$\|X - Y\|_F^2 \geq \|(X - Y)\vec{w}\|_2^2 = \|X\vec{w}\|_2^2 = \left\| \sum_{i=1}^{k+1} c_i \sigma_i \vec{u}_i \right\|_2^2 = \sum_{i=1}^{k+1} c_i^2 \sigma_i^2 \geq \sum_{i=1}^{k+1} c_i^2 \sigma_{k+1}^2 = \sigma_{k+1}^2. \quad (32)$$

More carefully, iterating this argument for each discarded component gives $\|X - Y\|_F^2 \geq \sum_{i>k} \sigma_i^2$. Meanwhile, the truncated SVD X_k achieves equality:

$$\|X - X_k\|_F^2 = \left\| \sum_{i=k+1}^r \sigma_i \vec{u}_i \vec{v}_i^T \right\|_F^2 = \sum_{i=k+1}^r \sigma_i^2, \quad (33)$$

using the fact that the Frobenius norm of a sum of rank-1 matrices with orthonormal columns and rows equals the sum of the squares of the coefficients. $\square$

Example 2.1 (Low-Rank Approximation of a Return Matrix). Suppose we observe monthly returns for $n = 500$ stocks over $T = 60$ months, giving a 60×500 data matrix X . Computing the SVD, we find that the first 3 singular values are $\sigma_1 = 42, \sigma_2 = 18, \sigma_3 = 11$, and the rest are below 5. The fraction of total variance explained by the rank-3 approximation is

$$\frac{\sigma_1^2 + \sigma_2^2 + \sigma_3^2}{\sum_i \sigma_i^2} = \frac{42^2 + 18^2 + 11^2}{42^2 + 18^2 + 11^2 + \dots} \approx 0.78. \quad (34)$$

Three factors explain 78% of the variation in 500 stocks. The columns of U_3 (the first 3 left singular vectors) are interpretable as the “market,” “size,” and “value” factor return series.

Proposition 2.1 (Maximum Variance). *The singular values are ordered with descending variance. If sample covariance of centered data is defined as*

$$\xi = \frac{1}{n} X^T X, \quad (35)$$

then $\max_{\|\vec{v}\|=1} \text{Var}(X\vec{v}) = \max_{\|\vec{v}\|=1} \vec{v}^T \xi \vec{v}$ has solution vector equal to the eigenvector with the largest eigenvalue, and has a solution equal to that eigenvalue. Hence, the first SVD vector points in the direction of greatest data variance.

Proof. The problem $\max_{\|\vec{v}\|=1} \vec{v}^T \xi \vec{v}$ is solved via the method of Lagrange multipliers. Define $\mathcal{L} = \vec{v}^T \xi \vec{v} - \mu(\vec{v}^T \vec{v} - 1)$. Setting $\nabla_{\vec{v}} \mathcal{L} = 0$ gives $2\xi \vec{v} = 2\mu \vec{v}$, i.e., $\vec{v}$ must be an eigenvector of ξ with eigenvalue μ . Substituting back, $\vec{v}^T \xi \vec{v} = \mu \vec{v}^T \vec{v} = \mu$, so the maximum is the largest eigenvalue λ_1 and the maximizer is the corresponding eigenvector $\vec{v}_1$. For subsequent components, imposing the additional constraint $\vec{v}^T \vec{v}_j = 0$ for $j < i$ at the i th step (which adds Lagrange terms that vanish by orthogonality of eigenvectors) gives $\vec{v}_i$ as the i th eigenvector with value λ_i . $\square$

Theorem 2.3 (Best Estimator). *Let $\{(\vec{x}_i, y_i)\}$ be observed data, and consider estimators of y of form $f(\vec{x} \cdot \vec{\beta})$. Then the conditional expectation is the best unconditional predictor of y — that is,*

$$\mathbb{E}[y|x] = \arg \min_{f \in \mathbb{F}} \mathbb{E}[(y - f(x))^2], \quad (36)$$

for the class of measurable functions.

Proof. Via the tower property,

$$\mathbb{E}[(y - f(x))^2] = \mathbb{E}[\mathbb{E}[(y - f(x))^2|x]]. \quad (37)$$

Massaging the inside yields

$$(y - f)^2 = (y - \mathbb{E}[y|x] + \mathbb{E}[y|x] - f)^2 \quad (38)$$

$$= (y - \mathbb{E}[y|x])^2 + 2(y - \mathbb{E}[y|x])(\mathbb{E}[y|x] - f) + (\mathbb{E}[y|x] - f)^2. \quad (39)$$

Taking the conditional expectation gives

$$\mathbb{E}[(y - f(x))^2|x] = \mathbb{E}[(y - \mathbb{E}[y|x])^2|x] + (\mathbb{E}[y|x] - f)^2, \quad (40)$$

which is minimized if $f = \mathbb{E}[y|x]$. Therefore, this choice pointwise minimizes the unconditional case, and is the unconditional minimum. $\square$

The Best Estimator theorem establishes that $\mathbb{E}[y|\vec{x}]$ is the optimal predictor under squared error loss, but says nothing about how well we can estimate the parameters governing that conditional expectation from finite data. The asymptotic theory of maximum likelihood estimation fills this gap: it characterizes the sampling distribution of $\hat{\theta}_{MLE}$ as $n \rightarrow \infty$, establishing both consistency and an efficiency benchmark that no regular estimator can beat.

Definition 2.1 (Likelihood and Score). Let $\{y_i\}_{i=1}^n$ be an i.i.d. sample from a parametric family $\{p(y; \theta) : \theta \in \Theta \subseteq \mathbb{R}^k\}$. The *log-likelihood* is

$$\ell(\theta) = \sum_{i=1}^n \log p(y_i; \theta). \quad (41)$$

The *score function* is the gradient of the log-likelihood,

$$s(\theta) = \nabla_{\theta} \ell(\theta) = \sum_{i=1}^n \nabla_{\theta} \log p(y_i; \theta). \quad (42)$$

The *maximum likelihood estimator* (MLE) is $\hat{\theta}_{MLE} = \arg \max_{\theta \in \Theta} \ell(\theta)$, which satisfies the first-order condition $s(\hat{\theta}) = 0$ at an interior solution.

Definition 2.2 (Fisher Information). The *Fisher information matrix* at θ is

$$\mathcal{I}(\theta) = \mathbb{E}[(\nabla_{\theta} \log p(y; \theta)) (\nabla_{\theta} \log p(y; \theta))^T] = -\mathbb{E}[\nabla_{\theta}^2 \log p(y; \theta)], \quad (43)$$

where the equality between the two expressions holds under regularity conditions permitting differentiation under the integral sign to obtain the Hessian. The Fisher information measures the curvature of the expected log-likelihood: high curvature means the data are highly informative about θ , so the likelihood surface has a sharp peak and $\hat{\theta}$ is precisely estimated.

Proof of equivalence of the two expressions. Since $\int p(y; \theta) dy = 1$, differentiating twice with respect to θ and exchanging differentiation and integration gives

$$0 = \int \nabla_{\theta} p(y; \theta) dy = \int p(y; \theta) \nabla_{\theta} \log p(y; \theta) dy, \quad (44)$$

so $\mathbb{E}[\nabla_{\theta} \log p(y; \theta)] = 0$: the score has mean zero at the true θ . Differentiating the score expectation once more,

$$0 = \int \nabla_{\theta} [p(y; \theta) \nabla_{\theta} \log p(y; \theta)] dy = \mathbb{E}[\nabla_{\theta}^2 \log p(y; \theta)] + \mathbb{E}[\nabla_{\theta} \log p(y; \theta) \nabla_{\theta} \log p(y; \theta)'], \quad (45)$$

which rearranges to the stated equality. $\square$

Theorem 2.4 (Cramér–Rao Lower Bound). *Let $\tilde{\theta}$ be any unbiased estimator of θ . Then*

$$\text{Var}(\tilde{\theta}) \geq \mathcal{I}(\theta)^{-1}, \quad (46)$$

in the sense that the matrix $\text{Var}(\tilde{\theta}) - \mathcal{I}(\theta)^{-1}$ is positive semi-definite. No unbiased estimator can have variance smaller than the inverted Fisher matrix $\mathcal{I}(\theta)^{-1}$.

Proof. For any unbiased estimator $\tilde{\theta}$ and any vector $\vec{a} \in \mathbb{R}^k$, consider the scalar $T = \vec{a} \cdot \tilde{\theta}$. By the Cauchy–Schwarz inequality applied to the covariance between T and the score $s(\theta) = \sum_i \nabla_{\theta} \log p(y_i; \theta)$,

$$\text{Cov}(T, s(\theta))^2 \leq \text{Var}(T) \text{Var}(s(\theta)). \quad (47)$$

Since $\tilde{\theta}$ is unbiased, $\mathbb{E}[\tilde{\theta}] = \theta$ for all θ , so differentiating under the integral gives $\text{Cov}(\tilde{\theta}, s(\theta)) = I_k$ (the $k \times k$ identity). Hence $\text{Cov}(T, s(\theta)) = \vec{a}' I_k = \vec{a}'$, and $\text{Var}(s(\theta)) = n \mathcal{I}(\theta)$. Substituting,

$$(\vec{a}' \vec{a})^2 \leq \text{Var}(T) \cdot n \vec{a}' \mathcal{I}(\theta) \vec{a}. \quad (48)$$

Since $\text{Var}(T) = \vec{a}' \text{Var}(\tilde{\theta}) \vec{a}$, dividing through by $n \vec{a}' \mathcal{I}(\theta) \vec{a}$ gives $\vec{a}' \text{Var}(\tilde{\theta}) \vec{a} \geq \frac{1}{n} \vec{a}' \mathcal{I}(\theta)^{-1} \vec{a}$. Since $\vec{a}$ was arbitrary, $\text{Var}(\tilde{\theta}) \geq \frac{1}{n} \mathcal{I}(\theta)^{-1}$. $\square$

Proposition 2.2 (Asymptotic Normality of the MLE). *Under regularity conditions, as $n \rightarrow \infty$,*

$$\sqrt{n}(\hat{\theta}_{MLE} - \theta_0) \xrightarrow{d} \mathcal{N}(0, \mathcal{I}(\theta_0)^{-1}). \quad (49)$$

The MLE is consistent ($\hat{\theta} \xrightarrow{p} \theta_0$) and asymptotically efficient (its asymptotic variance achieves the Cramér–Rao lower bound).

Proof sketch. Taylor-expand the score around the true parameter θ_0 :

$$0 = s(\hat{\theta}) = s(\theta_0) + \nabla_{\theta} s(\theta_0)(\hat{\theta} - \theta_0) + o_p(\|\hat{\theta} - \theta_0\|), \quad (50)$$

where $s(\hat{\theta}) = 0$ by the first-order condition. Rearranging,

$$\sqrt{n}(\hat{\theta} - \theta_0) = \left[-\frac{1}{n} \nabla_{\theta} s(\theta_0) \right]^{-1} \frac{s(\theta_0)}{\sqrt{n}} + o_p(1). \quad (51)$$

By the law of large numbers, $-\frac{1}{n} \nabla_{\theta} s(\theta_0) = \frac{1}{n} \sum_i [-\nabla_{\theta}^2 \log p(y_i; \theta_0)] \xrightarrow{p} \mathcal{I}(\theta_0)$. By the central limit theorem, since the individual scores $\nabla_{\theta} \log p(y_i; \theta_0)$ are i.i.d. with mean zero and variance $\mathcal{I}(\theta_0)$,

$$\frac{s(\theta_0)}{\sqrt{n}} = \frac{1}{\sqrt{n}} \sum_{i=1}^n \nabla_{\theta} \log p(y_i; \theta_0) \xrightarrow{d} \mathcal{N}(0, \mathcal{I}(\theta_0)). \quad (52)$$

Combining via Slutsky's theorem gives $\sqrt{n}(\hat{\theta} - \theta_0) \xrightarrow{d} \mathcal{N}(0, \mathcal{I}(\theta_0)^{-1})$. $\square$

Remark 2.2 (Hypothesis Testing). The asymptotic normality of the MLE generates a framework for hypothesis testing. For a null hypothesis $H_0 : R\theta_0 = r$ (where R is a $q \times k$ restriction matrix), three asymptotically equivalent tests with the same χ_q^2 null distribution are available:

- **Wald test:** $(R\hat{\theta} - r)'[R\hat{\mathcal{I}}^{-1}R'/n]^{-1}(R\hat{\theta} - r) \xrightarrow{d} \chi_q^2$. Requires only the unrestricted MLE $\hat{\theta}$; this is the test implicitly used when reporting t -statistics on regression coefficients (the scalar case $q = 1$ gives $t = \hat{\theta}_j / \widehat{\text{se}}(\hat{\theta}_j) \xrightarrow{d} \mathcal{N}(0, 1)$).
- **Likelihood ratio test:** $2[\ell(\hat{\theta}) - \ell(\tilde{\theta})] \xrightarrow{d} \chi_q^2$, where $\tilde{\theta}$ is the restricted MLE. Requires fitting both models; invariant to reparametrization.
- **Score (Lagrange multiplier) test:** $s(\tilde{\theta})'\hat{\mathcal{I}}^{-1}s(\tilde{\theta})/n \xrightarrow{d} \chi_q^2$. Requires only the restricted MLE $\tilde{\theta}$; useful when the unrestricted model is expensive to fit.

In the linear regression setting, all three tests coincide in finite samples under Gaussian errors and yield the familiar F -test. For logistic regression, where the OLS F -test is not available, the likelihood ratio and Wald tests are the standard tools.

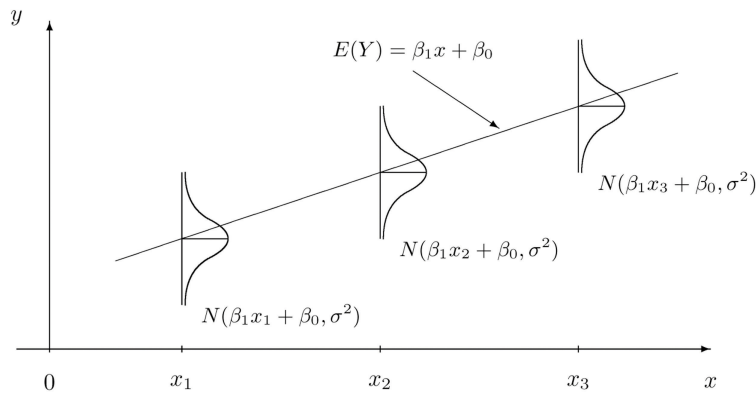


Figure 3: Gaussian linear regression

Definition 2.3 (Gaussian Linear Regression). Call a *linear regression* a model of the form

$$y = \vec{x} \cdot \vec{\beta} + \varepsilon. \quad (53)$$

The model is said to be a *Gaussian linear regression* if

$$y|\vec{x} \sim \mathcal{N}(\vec{x} \cdot \vec{\beta}, \sigma^2) \quad (54)$$

for homoskedastic error.

Proposition 2.3 (Maximum Entropy of the Gaussian). *Among all continuous distributions on $\mathbb{R}$ with fixed mean μ and variance σ^2 , the Gaussian $\mathcal{N}(\mu, \sigma^2)$ uniquely maximizes the differential entropy*

$$h(p) = - \int_{-\infty}^{\infty} p(x) \log p(x) dx. \quad (55)$$

Proof. Let p be any density with mean μ and variance σ^2 , and let $\phi = \mathcal{N}(\mu, \sigma^2)$ be the Gaussian with the same moments. The non-negativity of KL divergence gives

$$0 \leq D_{\text{KL}}(p \parallel \phi) = \int p(x) \log \frac{p(x)}{\phi(x)} dx = -h(p) - \int p(x) \log \phi(x) dx. \quad (56)$$

Since $\log \phi(x) = -\frac{1}{2} \log(2\pi\sigma^2) - \frac{(x-\mu)^2}{2\sigma^2}$, and both p and ϕ share the same mean and variance,

$$\int p(x) \log \phi(x) dx = -\frac{1}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \int p(x)(x-\mu)^2 dx = -\frac{1}{2} \log(2\pi\sigma^2) - \frac{1}{2} = \int \phi(x) \log \phi(x) dx, \quad (57)$$

which is $-h(\phi)$. Substituting back into the KL inequality gives $h(p) \leq h(\phi)$, with equality if and only if $p = \phi$ almost everywhere. $\square$

Remark 2.3 (Choice of Noise Model). The maximum entropy result gives the Gaussian noise assumption its privileged status. Suppose the true data-generating process is $y = \vec{x} \cdot \vec{\beta} + \varepsilon$, and all that we know about ε is its mean (zero) and variance (σ^2). The maximum entropy principle instructs us to model ε as Gaussian: it is the distribution that encodes exactly those two constraints and nothing else, injecting no additional structure beyond what the data support. Any other choice of noise model implicitly asserts additional knowledge about the error distribution's shape.

Under Gaussian noise, the log-likelihood of the sample is

$$\ell(\vec{\beta}) = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \vec{x}_i \cdot \vec{\beta})^2, \quad (58)$$

so maximizing the likelihood is exactly equivalent to minimizing the sum of squared residuals. The Gaussian assumption is therefore the theoretical justification for least squares as the default loss: ordinary least squares is MLE under the noise model that makes the fewest assumptions given fixed variance.

Different assumptions about the error distribution yield different loss functions. The following table summarizes the correspondence, which unifies the regression methods covered throughout these notes:

Noise model	Density $p(\varepsilon)$	MLE loss	Method
Gaussian	$\propto \exp\left(-\frac{\varepsilon^2}{2\sigma^2}\right)$	$\sum (y_i - \hat{y}_i)^2$	OLS, Ridge, LASSO
Laplace	$\propto \exp\left(-\frac{ \varepsilon }{\sigma}\right)$	$\sum y_i - \hat{y}_i $	Least absolute deviations
Bernoulli	$p^y(1-p)^{1-y}, p = \sigma(\vec{x} \cdot \vec{\beta})$	Cross-entropy	Logistic regression
Poisson	$\frac{\lambda^y e^{-\lambda}}{y!}, \lambda = e^{\vec{x} \cdot \vec{\beta}}$	Poisson deviance	Log-linear GLM
Asymmetric Laplace	$\propto \exp\left(-\frac{\varepsilon(\tau - \mathbf{1}_{\varepsilon < 0})}{\sigma}\right)$	Pinball loss	Quantile regression

Table 1: Noise model, implied loss function, and regression method.

Several points are worth noting. First, the Laplace distribution has heavier tails than the Gaussian, so MLE under Laplace noise penalizes large residuals less aggressively, producing an estimator robust to outliers. The Huber loss used in boosted regression trees (discussed later) interpolates between the two: quadratic for small residuals (Gaussian regime) and linear for large ones (Laplace regime). Second, the Poisson model is appropriate when y is a non-negative integer count (trade arrivals, corporate defaults, or news article counts per period) with variance proportional to the mean. Third, the cross-entropy loss of logistic regression is the negative log-likelihood of the Bernoulli model, confirming that logistic regression is MLE for binary outcomes under no additional distributional assumptions beyond the Bernoulli law. Fourth, quantile regression at level $\tau \in (0, 1)$ is MLE under an asymmetric Laplace distribution, providing the natural frequentist counterpart to the Bayesian interpretation of quantile estimation.

The entropy-maximization perspective also explains why OLS can perform well even when the true noise is non-Gaussian: the Gaussian MLE estimator (OLS) is consistent and asymptotically normal under any noise distribution with finite variance, by the central limit theorem. The loss of efficiency relative to the true MLE estimator is the price paid for not specifying the noise model correctly, but that loss is often modest in practice.

Proposition 2.4 (Best Linear Predictor). *Linear regression is the best linear predictor if exogeneity is satisfied. In that case,*

$$\hat{\beta} = (X^T X)^{-1} X^T \vec{y}. \quad (59)$$

This is known as the ordinary least-squares (OLS) estimator.

Proof. Identifying the minimizer of squared error $\min_{\beta} \mathbb{E}[(y - \vec{x} \cdot \vec{\beta})^2]$ is accomplished via setting the gradient to 0:

$$\nabla_{\beta} \mathbb{E}[(y - \vec{x} \cdot \vec{\beta})^2] = -2\mathbb{E}[\vec{x}(y - \vec{x} \cdot \vec{\beta})] = 0 \quad (60)$$

$$\mathbb{E}[\vec{x}\varepsilon] = 0. \quad (61)$$

Hence, this is accomplished if the errors are exogenous to the features. In that case, for samples,

$$\mathbb{E}_n[\vec{x}_i(y - \vec{x}_i \cdot \vec{\beta})] = 0 \quad \Rightarrow \quad \hat{\beta} = \mathbb{E}_n[\vec{x}_i \vec{x}_i^T]^{-1} \mathbb{E}_n[\vec{x}_i^T y_i]. \quad (62)$$

The second-order optimization condition is satisfied by checking that the Hessian is positive-definite. $\square$

Example 2.2 (OLS for CAPM Estimation). To estimate the CAPM beta for a stock, regress its monthly excess return on the market's excess return. Suppose we have $T = 60$ months of data and obtain

$$\hat{\beta} = \frac{\sum_{t=1}^{60} (R_{m,t} - \bar{R}_m)(R_{i,t} - \bar{R}_i)}{\sum_{t=1}^{60} (R_{m,t} - \bar{R}_m)^2} = 1.15, \quad \hat{\alpha} = 0.3\%/\text{month}. \quad (63)$$

Testing $H_0 : \alpha = 0$ via $t = \hat{\alpha}/\text{SE}(\hat{\alpha})$ determines whether the stock earns abnormal returns not explained by market exposure. If $\text{SE}(\hat{\alpha}) = 0.2\%$, then $t = 1.5$, which is not significant at the 5% level, hence the CAPM would price this stock adequately.

Theorem 2.5 (Gauss–Markov). *Consider the linear regression model*

$$\vec{y} = X\vec{\beta} + \vec{\varepsilon}, \quad (64)$$

where $\vec{y} \in \mathbb{R}^n$, $X \in \mathbb{R}^{n \times k}$ has full column rank, and the error vector satisfies

$$E[\vec{\varepsilon}] = 0, \quad \text{Var}(\vec{\varepsilon}) = \sigma^2 I_n. \quad (65)$$

Then the OLS estimator

$$\hat{\beta} = (X^T X)^{-1} X^T \vec{y} \quad (66)$$

is the best linear unbiased estimator (BLUE) of $\vec{\beta}$ — that is, no other linear unbiased estimator has lower variance.

Proof. Consider another estimator $\tilde{\beta} = C\vec{y} = ((X^T X)^{-1} X^T + D)\vec{y}$, for some matrix D . Then

$$\mathbb{E}[\tilde{\beta}] = \mathbb{E}[(X^T X)^{-1} X^T + D](X\vec{\beta} + \vec{\varepsilon}) \quad (67)$$

$$= (X^T X)^{-1} X^T X\vec{\beta} + DX\vec{\beta} \quad (68)$$

$$= (I_k + DX)\vec{\beta}, \quad (69)$$

so $DX\vec{\beta}$ must be 0 to not bias the estimator (note that $D = 0$ gives the usual OLS estimator,

which is unbiased). Then,

$$\text{Var}(\tilde{\beta}) = \text{Var}(C\vec{y}) \quad (70)$$

$$= \mathbb{E}[(C(\vec{y} - \mathbb{E}[\vec{y}]))(C(\vec{y} - \mathbb{E}[\vec{y}]))^T] \quad (71)$$

$$= C\text{Var}(\vec{y})C^T \quad (72)$$

$$= \sigma^2 CC^T \quad (73)$$

$$= \sigma^2((X^T X)^{-1} X^T + D)(X(X^T X)^{-1} + D^T) \quad (74)$$

$$= \sigma^2(X^T X)^{-1} + \sigma^2 DD^T \quad (75)$$

$$= \text{Var}(\hat{\beta}) + \sigma^2 DD^T. \quad (76)$$

Since DD^T is positive semi-definite, the OLS variance is exceeded by a positive semi-definite matrix. $\square$

Example 2.3 (Fisher Information for Gaussian Linear Regression). In the Gaussian linear regression model $y_i = \vec{x}_i \cdot \vec{\beta} + \varepsilon_i$, $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$, the log-likelihood for a single observation is

$$\log p(y_i; \vec{\beta}) = -\frac{1}{2} \log(2\pi\sigma^2) - \frac{(y_i - \vec{x}_i \cdot \vec{\beta})^2}{2\sigma^2}. \quad (77)$$

The score with respect to $\vec{\beta}$ is $\nabla_{\vec{\beta}} \log p = \frac{1}{\sigma^2} \vec{x}_i (y_i - \vec{x}_i \cdot \vec{\beta})$, and the Hessian is $\nabla_{\vec{\beta}}^2 \log p = -\frac{1}{\sigma^2} \vec{x}_i \vec{x}_i^T$. Taking expectations (noting that the Hessian is non-random here),

$$\mathcal{I}(\vec{\beta}) = \frac{1}{\sigma^2} \mathbb{E}[\vec{x}_i \vec{x}_i^T] = \frac{1}{\sigma^2} \Sigma_x, \quad (78)$$

where $\Sigma_x = \mathbb{E}[\vec{x}_i \vec{x}_i^T]$ is the second-moment matrix of the regressors. The Cramér–Rao bound is therefore $\text{Var}(\hat{\vec{\beta}}) \geq \frac{\sigma^2}{n} \Sigma_x^{-1}$, and the OLS estimator $\hat{\vec{\beta}} = (X'X)^{-1} X' \vec{y}$ achieves this bound exactly in finite samples: $\text{Var}(\hat{\vec{\beta}}) = \sigma^2 (X'X)^{-1} \approx \frac{\sigma^2}{n} \Sigma_x^{-1}$ for large n . This confirms that OLS is fully efficient under Gaussian errors, and not merely BLUE.

Remark 2.4 (MLE, OLS, and Gauss–Markov). Gauss–Markov establishes that OLS is the Best Linear Unbiased Estimator (BLUE): no other *linear* unbiased estimator has lower variance. The Cramér–Rao bound is a strictly stronger statement: under Gaussian noise, OLS is MLE, and MLE achieves the lowest variance among *all* regular unbiased estimators, not just linear ones. Gauss–Markov requires only $\mathbb{E}[\vec{\varepsilon}] = 0$ and $\text{Var}(\vec{\varepsilon}) = \sigma^2 I_n$; the MLE efficiency result additionally requires the Gaussian distributional assumption. The two results are therefore complementary: Gauss–Markov is distribution-free but restricted to linear estimators; MLE efficiency is fully efficient but requires correct specification of the noise distribution.

Definition 2.4 (Multicollinearity). For data matrix X , with the i th column being the vector i th covariate’s value for each data sample,

$$X = [\vec{x}_1 \dots \vec{x}_n], \quad (79)$$

then two regressors being perfectly correlated is referred to as *multicollinearity*. In that case, the columns of X are linearly dependent, so $X^T X$ is non-invertible, and $\hat{\beta}$ does not exist. If it is almost singular, the entries in the inverse blow up.

Example 2.4 (Multicollinearity in Practice). Suppose we regress stock returns on both the 3-month and 6-month Treasury rates as separate features. Because these rates are highly correlated ($\rho \approx 0.97$), $X^T X$ is nearly singular. The OLS estimator exists but has enormous variance: the estimated coefficients fluctuate wildly across subsamples. The variance inflation factor $VIF_j = 1/(1 - R_j^2)$, where R_j^2 is the R^2 from regressing covariate j on all others, exceeds 15 for both rate variable, a common diagnostic for problematic multicollinearity. Ridge regression (discussed later) or dropping one of the rates resolves this.

Definition 2.5 (Logistic Regression). Linear regression is ill-suited for binary outcomes: the linear predictor $\vec{x}_i \cdot \vec{\beta}$ is unbounded, whereas a probability must lie in $[0, 1]$. *Logistic regression* resolves this. Consider $\{(\vec{x}_i, Y_i)\}$, where the output $Y \in \{0, 1\}$, for classification, is distributed $Y_i | \vec{x}_i \sim \text{Bernoulli}(p_i)$. Then

$$\mathbb{P}(Y_i = y | \vec{x}_i) = p_i^y (1 - p_i)^{1-y}. \quad (80)$$

Define p_i with a *sigmoid* function:

$$p_i = \frac{1}{1 + e^{-\vec{x}_i \cdot \vec{\beta}}}. \quad (81)$$

The likelihood of observing the data is then:

$$\mathcal{L}(\vec{\beta}) = \prod_i p_i^{Y_i} (1 - p_i)^{1-Y_i} \quad (82)$$

$$l(\vec{\beta}) = \sum_i \left[Y_i \vec{x}_i \cdot \vec{\beta} - \log(1 + e^{\vec{x}_i \cdot \vec{\beta}}) \right] \quad (83)$$

$$\nabla_{\vec{\beta}} l(\vec{\beta}) = \sum_i \vec{x}_i \left(Y_i - \frac{1}{1 + e^{-\vec{x}_i \cdot \vec{\beta}}} \right) = X^T (\vec{y} - \sigma(X\vec{\beta})) = 0. \quad (84)$$

Remark 2.5. The decision boundary occurs where $\vec{x}_i \cdot \vec{\beta} = 0$, as that is where $p = 1/2$ occurs. The maximum likelihood solution has no closed form, since the gradient equation is non-linear, and requires an iterative method such as gradient descent. Gradient descent updates parameters in the direction of the negative gradient of the loss:

$$\vec{\beta}^{(t+1)} = \vec{\beta}^{(t)} + \eta \nabla_{\vec{\beta}} l(\vec{\beta}^{(t)}), \quad (85)$$

where $\eta > 0$ is the learning rate. Convergence to the global maximum is guaranteed here since the log-likelihood is concave.

Example 2.5 (Credit Default Prediction). A bank models the probability of loan default ($Y = 1$) using three features: debt-to-income ratio (x_1), credit score (x_2), and log loan amount (x_3). After

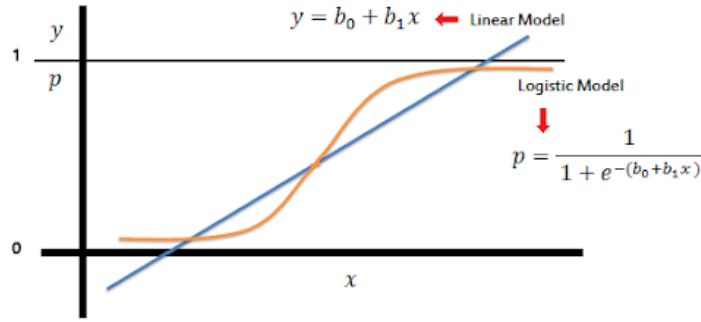


Figure 4: Logistic and linear regression

fitting logistic regression on historical data, the estimated coefficients are $\hat{\beta}_1 = 2.1$, $\hat{\beta}_2 = -0.03$, $\hat{\beta}_3 = 0.4$, $\hat{\beta}_0 = 15.5$. For a borrower with $x_1 = 0.45$, $x_2 = 680$, $x_3 = \ln(250,000) = 12.43$:

$$\vec{x} \cdot \vec{\hat{\beta}} = 15.5 + 2.1(0.45) - 0.03(680) + 0.4(12.43) = -0.678, \quad (86)$$

$$\hat{p} = \frac{1}{1 + e^{0.678}} \approx 0.337. \quad (87)$$

The bank predicts a 33.7% default probability. A higher debt-to-income ratio or lower credit score pushes the linear predictor positive, increasing the estimated default probability.

Definition 2.6 (Deviance). For a model with log-likelihood as a function of parameters, and an observed dataset, define deviance as a goodness-of-fit measure:

$$\text{Dev}(\vec{\beta}) = -2(l(\vec{\beta}) - l(\text{saturated})), \quad (88)$$

where the saturated model sets one parameter per data point. Minimizing deviance is hence equivalent to maximizing likelihood.

Definition 2.7 (Model R^2). Define the *residual deviance* $D = \text{Dev}(\vec{\beta})$ of a model as the deviance where the prediction comes from $\vec{x} \cdot \vec{\beta}$, and define the *null deviance* $D_0 = \text{Dev}(\hat{y} = \bar{y})$ as the deviance where the prediction is replaced by the average of the response data. Call the following value R^2 , a measure of the improvement in explanation of the data over the naive model:

$$R^2 = 1 - \frac{D}{D_0}. \quad (89)$$

For linear regression, this is

$$R^2 = 1 - \frac{\sum_i (y_i - \vec{x}_i \cdot \hat{\beta})^2}{\sum_i (y_i - \bar{y})^2} = 1 - \frac{\text{RSS}}{\text{TSS}}. \quad (90)$$

Remark 2.6 (OOS R^2 in Finance). In stock return prediction, in-sample R^2 values of 5–10% are common, but the relevant metric is out-of-sample (OOS) R^2 . An OOS R^2 of just 0.5% per month

for individual stock returns is economically significant and can generate substantial portfolio profits. This starkly contrasts other domains where $R^2 < 30\%$ would be considered a poor model. The reason is that even tiny predictability, compounded over many assets and time periods with proper portfolio construction, translates into meaningful Sharpe ratios.

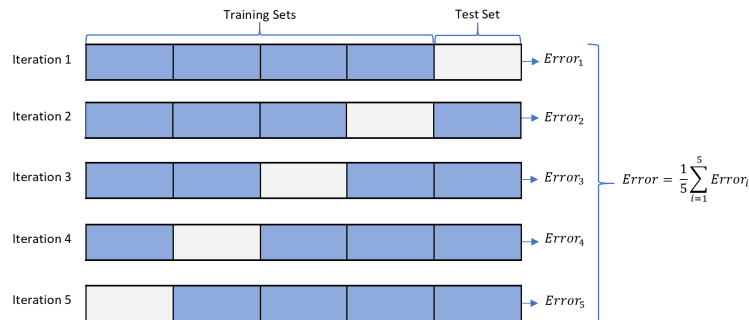


Figure 5: K-fold cross-validation

Definition 2.8 (K-Fold Cross-Validation). Another method of validating a model consists in splitting the dataset into k folds (subsections) and training on $k - 1$ folds while testing on the remaining fold by computing MSE. This “leave one out” is repeated for each fold, and the average MSE is given as the validation error.

3 Model Selection

Given a set of outcomes and a large amount of data per outcome, how to select variables to explain the outcomes? What is the best subset of covariates that predicts the outcomes? There are 2^Ω possibilities to check, yet this is often too costly to use.

Proposition 3.1. *Adding regressor covariates to a model will increase or keep constant the R^2 .*

Proof. If $y = x_1\beta_1 + \varepsilon$ has a certain RSS, consider adding to the model to get $y = x_1\beta_1 + x_2\beta_2 + \varepsilon$, which will not change TSS. If $\beta_2 \neq 0$ gives a better fit, then $\text{RSS}' < \text{RSS}$. If it does not, then the model will set $\beta_2 = 0$, and $\text{RSS}' = \text{RSS}$. $\square$

Definition 3.1 (Screening). Call *screening* the method whereby covariates are selected by inspecting their correlation with the output variable, and selecting those with correlation above a certain threshold. Notably, this process does not address multicollinearity of regressors.

Definition 3.2 (Information Criteria). Define the *Akaike information criterion* (*AIC*) as a measure of model fit that penalizes complexity:

$$\text{AIC} = 2k - 2 \ln(\hat{L}), \quad (91)$$

where k is the number of estimated parameters, and $\hat{L}$ is the maximized likelihood value. Likewise, define the *Bayesian information criterion (BIC)* as:

$$\text{BIC} = k \ln(n) - 2 \ln(\hat{L}), \quad (92)$$

where n is the number of data points.

Remark 3.1. BIC penalizes model complexity more heavily than AIC for $n \geq 8$ (since $\ln(n) > 2$), and therefore tends to select simpler models. In finance, BIC is often preferred for selecting the number of factors in a factor model because overfitting factors can lead to spurious in-sample performance that vanishes out of sample.

Definition 3.3 (Forward Stepwise). Rather than checking all subsets of regressors, begin with the null model, and iteratively check (model + $\beta_j x_j$) over remaining covariates for the minimum AIC, adding the best covariate until the next addition does not offer a lower model AIC. This is far faster than evaluating the entire power set, yet the MLE variance is non-negligible, so the “best model” might change with data resampling.

Theorem 3.1 (Karush-Kuhn-Tucker Conditions). *Consider the optimization problem*

$$\min_x f(x), \quad g_i(x) \leq 0, \quad h_j(x) = 0 \quad (93)$$

for arbitrary numbers of constraints. Define an objective function to be minimized

$$\mathcal{L} = f(x) + \sum_i \lambda_i g_i(x) + \sum_j \nu_j h_j(x). \quad (94)$$

Then at the optimal point x^ in a convex problem:*

1. (stationarity) $\nabla_x \mathcal{L} = 0$, as the objective cannot be locally improved.
2. (primal feasibility) $g_i(x^*) \leq 0, h_j(x^*) = 0$, as the constraints are obeyed.
3. (dual feasibility) $\lambda_i \geq 0$, as making $g > 0$ must increase the objective.
4. (complementary slackness) $\lambda_i g_i(x^*) = 0$, as constraint g_i must be active whenever the unconstrained minimizer lies outside the feasible region, as otherwise you could move further downhill while remaining feasible. If the constraint isn't forcing, then $\lambda = 0$, and this is unconstrained.

Proof sketch. Stationarity: at any interior optimum the gradient of the objective must vanish; at a boundary optimum the gradient of f must be a non-negative combination of the constraint gradients (otherwise one could move along the boundary to decrease f). Primal feasibility is the requirement that x^* is in the feasible set. Dual feasibility follows because the Lagrange multiplier for an inequality constraint measures the rate of change of the optimal value with respect to relaxing the constraint: if $\lambda_i < 0$, tightening g_i would decrease f , contradicting optimality. Complementary slackness: if $g_i(x^*) < 0$ (the constraint is slack), then perturbing g_i locally does not affect the

optimum, so $\lambda_i = 0$; if $\lambda_i > 0$, the constraint must be active. Together, $\lambda_i g_i(x^*) = 0$. For a convex problem, KKT conditions are both necessary and sufficient for global optimality. $\square$

Definition 3.4 (LASSO). *LASSO regression* is a regularization to linear regression that imposes an ℓ^1 norm penalty on $\vec{\beta}$:

$$\min_{\vec{\beta}} \left(\|\vec{y} - X\vec{\beta}\|_2^2 \right), \quad \|\vec{\beta}\|_1 \leq c \quad (95)$$

as a constrained optimization problem, or

$$\min_{\vec{\beta}} \left(\|\vec{y} - X\vec{\beta}\|_2^2 + \lambda \|\vec{\beta}\|_1 \right) \quad (96)$$

as an unconstrained problem.

Remark 3.2. LASSO is not differentiable at 0, so there is no closed-form solution. Geometrically, this constrains $\vec{\beta}$ to lie within a hyper-diamond, whose corners lie on the coordinate axes. Because the RSS contours are ellipsoidal, the constrained minimum typically occurs at a corner, setting some coefficients exactly to zero. LASSO therefore performs *variable selection*: it produces sparse solutions, setting irrelevant regressors to exactly 0. So-called “LASSO paths” are used for variable selection by setting λ very large, then gradually decreasing it to watch which regressors “turn on” as LASSO deems them important.

Theorem 3.2 (LASSO Soft Thresholding). *For orthonormal design X ($X^T X = I$), let $\hat{\beta}_j^{\text{OLS}} = \vec{x}_j^T \vec{y}$ be the OLS estimator for the j th coefficient. Then the LASSO solution is given componentwise by the soft thresholding operator:*

$$\hat{\beta}_j^{\text{LASSO}} = \text{sign}(\hat{\beta}_j^{\text{OLS}}) \max\left(|\hat{\beta}_j^{\text{OLS}}| - \frac{\lambda}{2}, 0\right). \quad (97)$$

Coefficients with $|\hat{\beta}_j^{\text{OLS}}| \leq \lambda/2$ are set exactly to zero; the remaining coefficients are shrunk toward zero by $\lambda/2$.

Proof. With orthonormal design, the LASSO objective separates into independent scalar problems:

$$\min_{\beta_j} \left[(\hat{\beta}_j^{\text{OLS}} - \beta_j)^2 + \lambda |\beta_j| \right]. \quad (98)$$

For $\beta_j > 0$, the derivative is $-2(\hat{\beta}_j^{\text{OLS}} - \beta_j) + \lambda = 0$, giving $\beta_j = \hat{\beta}_j^{\text{OLS}} - \lambda/2$, which is valid only if $\hat{\beta}_j^{\text{OLS}} > \lambda/2$. Similarly for $\beta_j < 0$. If $|\hat{\beta}_j^{\text{OLS}}| \leq \lambda/2$, the minimum occurs at $\beta_j = 0$ (verified by checking the subdifferential condition $0 \in \partial_{\beta_j} \mathcal{L}$, which requires $|2\hat{\beta}_j^{\text{OLS}}| \leq \lambda$). Combining the cases gives the soft thresholding operator. $\square$

Example 3.1 (LASSO for Factor Selection). A researcher wants to predict monthly stock returns using 50 candidate firm characteristics (size, value, momentum, profitability, investment, etc.). With λ chosen by 5-fold cross-validation, LASSO selects 8 nonzero coefficients: size ($\hat{\beta} = -0.12$),

book-to-market (0.09), momentum (0.15), profitability (0.08), investment (-0.06), short-term reversal (-0.11), idiosyncratic volatility (-0.14), and earnings surprise (0.07). The remaining 42 characteristics receive exactly zero coefficients. This sparse model achieves higher OOS R^2 than the full 50-variable OLS model, which overfits and has negative OOS R^2 .

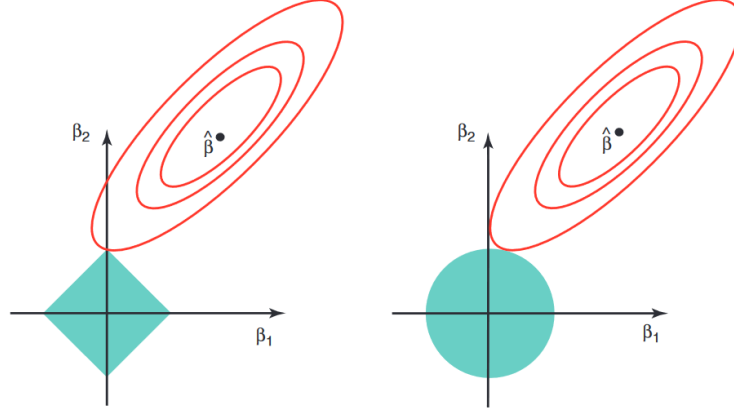


Figure 6: LASSO and Ridge error surfaces

Definition 3.5 (Ridge). *Ridge regression* is a regularization to linear regression that imposes an ℓ^2 norm penalty on $\vec{\beta}$:

$$\min_{\vec{\beta}} \left(\|\vec{y} - X\vec{\beta}\|_2^2 \right), \quad \|\vec{\beta}\|_2 \leq c \quad (99)$$

as a constrained optimization problem, or

$$\min_{\vec{\beta}} \left(\|\vec{y} - X\vec{\beta}\|_2^2 + \lambda \|\vec{\beta}\|_2^2 \right) \quad (100)$$

as an unconstrained problem.

Remark 3.3. Geometrically, this constrains $\vec{\beta}$ to a hyper-sphere. Unlike LASSO, the sphere has no corners, so the constrained minimum generically occurs at a smooth point on the boundary where all coordinates are nonzero. Ridge therefore *shrinks* all coefficients proportionally toward zero but never sets any exactly to zero. It does not perform variable selection.

Theorem 3.3 (Ridge $\vec{\beta}$). *The $\vec{\beta}$ for Ridge is*

$$\hat{\beta} = (X^T X + \lambda I)^{-1} X^T \vec{y}. \quad (101)$$

Note that even when $X^T X$ is singular, the λI term allows for matrix inversion, making Ridge a natural remedy for multicollinearity.

Proof. Take the derivative of the objective function to minimize:

$$\mathcal{L}(\vec{\beta}) = (\vec{y} - X\vec{\beta})^T(\vec{y} - X\vec{\beta}) + \lambda \vec{\beta}^T \vec{\beta} \quad (102)$$

$$\nabla_{\vec{\beta}} \mathcal{L}(\vec{\beta}) = -2X^T \vec{y} + 2X^T X \vec{\beta} + 2\lambda I \vec{\beta} = 0. \quad (103)$$

Rearranging gives the theorem. $\square$

Theorem 3.4 (Ridge Shrinkage in the SVD Basis). *Let $X = U\Sigma V^T$ be the SVD of the data matrix, with singular values $\sigma_1 \geq \dots \geq \sigma_p$. Then the Ridge prediction decomposes in the principal component basis as*

$$X \hat{\beta}^{\text{Ridge}} = \sum_{j=1}^p \frac{\sigma_j^2}{\sigma_j^2 + \lambda} \vec{u}_j (\vec{u}_j^T \vec{y}). \quad (104)$$

The shrinkage factor $d_j = \sigma_j^2 / (\sigma_j^2 + \lambda)$ satisfies $0 < d_j < 1$. Directions of large variance ($\sigma_j^2 \gg \lambda$) are barely shrunk ($d_j \approx 1$), while directions of small variance ($\sigma_j^2 \ll \lambda$) are heavily shrunk ($d_j \approx 0$). Ridge therefore performs soft dimensionality reduction: it nearly eliminates components where the signal-to-noise ratio is low.

Proof. Using $X = U\Sigma V^T$, we have $X^T X = V\Sigma^2 V^T$ and $X^T \vec{y} = V\Sigma U^T \vec{y}$. Then

$$\hat{\beta}^{\text{Ridge}} = (V\Sigma^2 V^T + \lambda I)^{-1} V\Sigma U^T \vec{y} = V(\Sigma^2 + \lambda I)^{-1} \Sigma U^T \vec{y}. \quad (105)$$

Multiplying by $X = U\Sigma V^T$:

$$X \hat{\beta}^{\text{Ridge}} = U\Sigma(\Sigma^2 + \lambda I)^{-1} \Sigma U^T \vec{y} = \sum_{j=1}^p \frac{\sigma_j^2}{\sigma_j^2 + \lambda} \vec{u}_j (\vec{u}_j^T \vec{y}). \quad (106)$$

Each $d_j = \sigma_j^2 / (\sigma_j^2 + \lambda) \in (0, 1)$ since both σ_j^2 and λ are positive. The monotonicity $d_j > d_{j+1}$ when $\sigma_j > \sigma_{j+1}$ follows directly. $\square$

Example 3.2 (Ridge vs. OLS with Multicollinear Regressors). Consider regressing bond returns on the first 3 principal components of the yield curve (level, slope, curvature) plus 7 macro variables that are highly correlated with these components. OLS on all 10 features gives wildly unstable estimates: rerunning on a slightly different sample period flips the sign of several coefficients. Ridge with λ chosen by cross-validation yields $d_j \approx 0.95$ for the first 3 directions (which carry most variance) but $d_j \approx 0.05$ for the last 4 directions (near-collinear combinations). Ridge effectively projects the data onto a roughly 6-dimensional subspace while keeping all 10 coefficients nonzero and stable.

Corollary 3.1 (Effective Degrees of Freedom). *One metric for effective degrees of freedom in regressions is the trace of the hat/projection matrix. The effective degrees of freedom of Ridge*

regression are

$$\text{df}(\lambda) = \text{tr} [X(X^T X + \lambda I)^{-1} X^T] = \sum_{j=1}^p \frac{\sigma_j^2}{\sigma_j^2 + \lambda}. \quad (107)$$

When $\lambda = 0$, $\text{df} = p$ (OLS). As $\lambda \rightarrow \infty$, $\text{df} \rightarrow 0$ (null model). This provides a continuous measure of model complexity governed by λ .

Definition 3.6 (Elastic Net). The *elastic net* combines the ℓ^1 and ℓ^2 penalties of LASSO and Ridge:

$$\min_{\beta} \left(\|\vec{y} - X\vec{\beta}\|_2^2 + \lambda_1 \|\vec{\beta}\|_1 + \lambda_2 \|\vec{\beta}\|_2^2 \right). \quad (108)$$

It inherits variable selection from LASSO while benefiting from the stability of Ridge in the presence of highly correlated predictors. The tuning parameters $\lambda_1, \lambda_2 \geq 0$ are selected by cross-validation; setting $\lambda_2 = 0$ recovers LASSO and $\lambda_1 = 0$ recovers Ridge.

Remark 3.4. In all regularized regression methods, the penalty parameters λ (or λ_1, λ_2 for the elastic net) govern the bias-variance tradeoff and are chosen by k -fold cross-validation: the value minimizing the average out-of-sample MSE across folds is selected.

Remark 3.5 (Regularization in Portfolio Construction). In mean-variance portfolio optimization, the optimal portfolio weights $\vec{w}^* = \Sigma^{-1} \vec{\mu}$ (up to scaling) are sensitive to estimation error in Σ and $\vec{\mu}$. Applying Ridge-type shrinkage to Σ^{-1} (or equivalently, shrinking the covariance eigenvalues toward their grand mean) produces more stable, better-performing portfolios. The Ledoit–Wolf shrinkage estimator $\hat{\Sigma}^{\text{LW}} = (1 - \alpha) \hat{\Sigma} + \alpha \bar{\lambda} I$ is the Ridge analogue for covariance estimation. LASSO-penalized portfolio weights enforce sparsity, producing portfolios that hold only a subset of available assets, reducing transaction costs.

4 Factor Models

To estimate $Y_t = \beta_0 + \beta_1 X_{1t} + \cdots + \beta_n X_{nt} + U_t$ for $t = 1, \dots, T$, or $Y = X\beta + U$ as matrices, if there are more features than data points ($n > T$), then the OLS solution fails.

Definition 4.1 (Factor Model). To address regression in high dimensions, reduce the dimension of the data set:

$$\vec{X}_t = \vec{\mu}_t + \Lambda \vec{F}_t + \vec{V}_t, \quad (109)$$

where $\vec{X}_t, \vec{\mu}_t$ are n -dimensional vectors, $\vec{F}_t$ is a $k \ll n$ -dimensional vector of factors, Λ is an unobserved $n \times k$ matrix of factor loadings, and $\vec{V}_t$ is an n -dimensional zero-mean vector of idiosyncratic shocks.

Definition 4.2 (Principal Component Analysis). Let $X = [\vec{X}_1 \cdots \vec{X}_T]$ be a data matrix of n features and T observations, $n > T$. In practice, X should first be centered (subtract the column

means) so that the PCA directions reflect genuine covariance structure rather than shifts in the mean. Call the covariance of centered data the $n \times n$ matrix $\hat{\Sigma} = \frac{1}{T}X^T X$. To find the k directions that explain the most variance of the data, find the vector maximizing $\text{Var}(X\vec{\gamma})$:

$$\max_{\vec{\gamma} \in \mathbb{R}^n} \vec{\gamma} \cdot \hat{\Sigma} \vec{\gamma}, \quad \|\vec{\gamma}\|_2 = 1 \quad (110)$$

$$\Rightarrow \nabla_{\vec{\gamma}}[\vec{\gamma} \cdot \hat{\Sigma} \vec{\gamma} - \lambda \vec{\gamma} \cdot \vec{\gamma}] = 0 \quad (111)$$

$$\hat{\Sigma} \vec{\gamma} = \lambda \vec{\gamma}. \quad (112)$$

So the eigenvector of the covariance matrix maximizes this, and its variance is the eigenvalue. Hence the first principal component is chosen as the covariance eigenvector with largest eigenvalue $\vec{\gamma}_1$. The other principal components are chosen as variance-maximizing directions orthogonal to the first, so the other highest eigenvectors are selected. Note that this is what SVD has been doing.

Taking k principal components, call $\Gamma_k = [\vec{\gamma}_1 \cdots \vec{\gamma}_k]$ an $n \times k$ matrix. Then the transformed dataset, as a $T \times k$ matrix, is

$$Z_k = X\Gamma_k = [\vec{z}_1 \cdots \vec{z}_k]. \quad (113)$$

The columns $X\vec{\gamma}_i$ then have $\vec{z}_i \cdot \vec{z}_j = \vec{\gamma}_i^T (\hat{\Sigma} \vec{\gamma}_j) = \delta_{ij}$, and $\text{Var}(\vec{z}_i) = \lambda_i$.

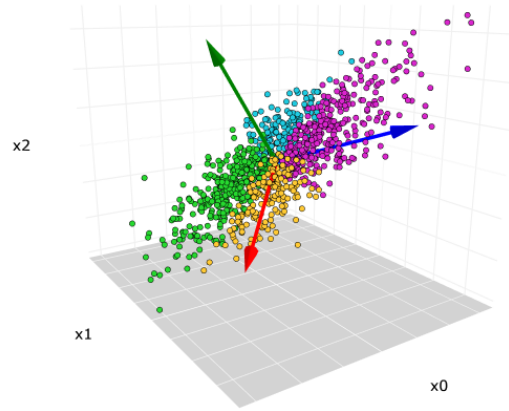


Figure 7: Principal component axes

Example 4.1 (PCA of the Treasury Yield Curve). Let X be a $T \times 10$ matrix of monthly changes in Treasury yields at maturities 1, 2, 3, 5, 7, 10, 15, 20, 25, and 30 years. PCA on the correlation matrix typically yields three dominant components:

1. **PC1 (“Level”)** explains $\sim 85\%$ of variance. The loadings are approximately equal across maturities: when PC1 moves, all yields shift in parallel. This corresponds to the overall interest rate level.
2. **PC2 (“Slope”)** explains $\sim 10\%$ of variance. Loadings are positive at the short end and negative at the long end: it captures steepening or flattening of the curve.

3. **PC3 (“Curvature”)** explains $\sim 3\%$ of variance. Loadings are positive at short and long maturities but negative in the middle: it captures “butterfly” movements.

Three factors explain $\sim 98\%$ of all yield curve variation. A bond portfolio manager thus hedges risk by neutralizing exposure to these three PCs rather than to each individual maturity.

Remark 4.1. To check what an appropriate value for k is, run a full $k = n$ PCA, and plot $\alpha_j = \frac{\lambda_j}{\sum_i \lambda_i}$ to check the relative proportion of variance that each PC explains. Alternatively, stop at the component after which there is the greatest drop. Alternatively, if PCA is being used in a model, an information criterion can be used.

Definition 4.3 (Identifiability). A parameter θ is *point identified* by a model P if

$$\theta_1 \neq \theta_2 \Rightarrow P_{\theta_1} \neq P_{\theta_2}. \quad (114)$$

If a model has *latent factors*, which are not immediately physical or observable, there can be issues with identifiability unless a constraint is chosen. This is the equivalent of having gauge freedom.

Theorem 4.1 (Factor Identification). *To have identifiability of parameters, the factor model $\vec{X}_t = \vec{\mu}_t + \Lambda \vec{F}_t + \vec{V}_t$ must have*

$$\mathbb{E}[\vec{F}_t] = 0, \quad \mathbb{E}[\vec{F}_t \vec{F}_t^T] = I_k, \quad \mathbb{E}[\vec{F}_t \vec{V}_t^T] = 0. \quad (115)$$

This gives

$$\Sigma_{\vec{X}_t} = \Lambda \Lambda^T + \mathbb{E}[\vec{V}_t \vec{V}_t^T]. \quad (116)$$

Proof. First, we can make $\vec{\mu}_t$ identifiable by setting $\mathbb{E}[\vec{F}_t] = 0$, in which case $\mathbb{E}[\vec{X}_t] = \vec{\mu}_t$. This holds WLOG, as we can pick $\tilde{F} = F - \mathbb{E}[F]$, which is zero mean. Specifying the other parameters is more difficult, since

$$\vec{X}_t = \vec{\mu}_t + \Lambda \vec{F}_t + \vec{V}_t = \vec{\mu}_t + (\Lambda H)(H^{-1} \vec{F}_t) + \vec{V}_t \quad (117)$$

for an invertible matrix H , hence $\Lambda, \vec{F}_t$ have freedom under linear transformation. Calculating the covariance of $\vec{X}_t$:

$$\Sigma = \Lambda \mathbb{E}[\vec{F}_t \vec{F}_t^T] \Lambda^T + \Lambda \mathbb{E}[\vec{F}_t \vec{V}_t^T] + \mathbb{E}[\vec{V}_t \vec{F}_t^T] \Lambda^T + \mathbb{E}[\vec{V}_t \vec{V}_t^T], \quad (118)$$

the covariance takes the above form if the constraints are imposed, which sets those terms to 0. $\square$

Theorem 4.2 (Factor Estimation). *The optimal choice of factors and loadings minimizes the total unexplained variance in the data. The solution is given by the principal components:*

$$\hat{F}_k = \sqrt{T} \Gamma_k(\Sigma), \quad \hat{\Lambda}_k = \frac{X \hat{F}_k}{T}, \quad (119)$$

where $\Sigma = X^T X / n$ and Γ_k gives the top k eigenvectors.

Proof. The optimal parameters minimize

$$q(\Lambda, F) = \sum_{i=1}^n \sum_{t=1}^T (X_{it} - \vec{\lambda}_i \cdot \vec{F}_t)^2 = \|X - \Lambda F^T\|_F^2. \quad (120)$$

If the factors are fixed, the solution is OLS $\hat{\lambda}_i = (F^T F)^{-1} F^T \vec{X}_i$. Imposing the constraint $\frac{F^T F}{T} = I_k$ gives

$$q(\Lambda, F) = \sum_{i=1}^n \sum_{t=1}^T \left(X_{it} - \frac{\vec{X}_i^T F}{T} \vec{F}_t \right)^2 = \left\| X \left(I_T - \frac{F F^T}{T} \right) \right\|_F^2. \quad (121)$$

This is minimizing the orthogonal complement from the projection onto factor space, the part of the data that the model cannot explain. Using the Frobenius norm,

$$\left\| X \left(I_T - \frac{F F^T}{T} \right) \right\|_F^2 = \text{tr} \left[\left(I_T - \frac{F F^T}{T} \right)^T X^T X \left(I_T - \frac{F F^T}{T} \right) \right] \quad (122)$$

$$= \text{tr} \left[\left(I_T - \frac{F F^T}{T} \right) X^T X \right], \quad (123)$$

exploiting the cyclicity of the trace and idempotency of the projection. So

$$\left\| X \left(I_T - \frac{F F^T}{T} \right) \right\|_F^2 = C - \frac{1}{T} \text{tr}[F^T X^T X F]. \quad (124)$$

Thus, the optimization problem is

$$\max_F \text{tr}[F^T X^T X F], \quad \frac{F^T F}{T} = I_k. \quad (125)$$

Rescaling to $U = \frac{F}{\sqrt{T}}, S = X^T X$, this is

$$\max_U \text{tr}[U^T S U], \quad U^T U = I_k. \quad (126)$$

This is maximizing $\sum_{j=1}^k \vec{u}_j \cdot S \vec{u}_j$ subject to $\vec{u}_i \cdot \vec{u}_j = \delta_{ij}$, which is equivalent to finding the first k principal components. This result gives the theorem, keeping track of coefficients. $\square$

Remark 4.2. When using this to predict with new data, project the new vector to the factor space $\tilde{f}_{t+1} = (\Lambda^T \Lambda)^{-1} \Lambda^T \vec{x}_{t+1}$, then run the new vector through the model.

Remark 4.3 (Statistical vs. Observable Factors in Finance). PCA-extracted factors are *statistical* (latent): they are linear combinations of returns with no direct economic interpretation. In contrast, the Fama–French factors are *observable*: they are returns on long-short portfolios constructed from firm characteristics. The two approaches are complementary. Kelly, Pruitt, and Su (2019) propose “Instrumented PCA” (IPCA), which estimates latent factors whose loadings are functions of observable characteristics, bridging the gap. In practice, a 5-factor IPCA model explains the

cross-section of equity returns as well as far more complex machine learning methods, while retaining economic interpretability.

5 Ensemble Methods

Definition 5.1 (Greedy Algorithm). A *greedy algorithm* is any algorithm that follows the problem-solving heuristic of making the locally optimal choice at each stage. A greedy heuristic can yield locally optimal solutions that approximate a globally optimal solution in a reasonable amount of time. A greedy algorithm tends to work well for the class of problems satisfying the following two properties:

1. (Greedy choice property) Whichever choice seems best at a given moment can be made and then (recursively) solve the remaining sub-problems. The choice made by a greedy algorithm may depend on choices made so far, but not on future choices or all the solutions to the subproblem. It iteratively makes one greedy choice after another, reducing each given problem into a smaller one. In other words, a greedy algorithm never reconsiders its choices. This is the main difference from dynamic programming.
2. (Optimal substructure) An optimal solution to the problem is constructible from optimal solutions to the sub-problems.

Definition 5.2 (Additive Model). An *additive model* takes the form

$$Y_t = f_M(X_t) = \sum_{m=1}^M \beta_m f_m(X_t) + U_t, \quad (127)$$

where U_t is random noise and f_m is a basis function or weak learner (it does not do considerably better than random guessing).

Definition 5.3 (Function Estimation Problem). For variable $\vec{Y}$ the observations and X the collection of feature observations, define the optimal function for some loss function as

$$f^* = \arg \min_f \mathbb{E}[L(\vec{Y} - f(X))]. \quad (128)$$

In practice, for finite numbers of observations, not distributions, this is finding the function that minimizes the empirical risk function:

$$f^* = \arg \min_f \frac{1}{T} \sum_{t=1}^T L(Y_t - f(\vec{X}_t)). \quad (129)$$

Definition 5.4 (Gradient Boosting). Given the above problem, restrict the class of functions to additive models. Construct the additive model sequentially:

$$f_m(X) = f_{m-1}(X) + \beta_m h_m(X). \quad (130)$$

At each step, greedily choose the new function to minimize the loss of the model on the data — that is, fit the new function to match the existing residuals:

$$h_m = \arg \min_h \sum_{t=1}^T L([Y_t - f_{m-1}(X_t)] - h(X_t)). \quad (131)$$

More generally, define the “pseudo-residuals” as

$$g_t^{(m)} = - \left[\frac{\partial L(Y_t, f(X_t))}{\partial f(X_t)} \right]_{f=f_{m-1}}, \quad (132)$$

since this resembles the residuals for MSE loss, and fit h to match this. To calculate the coefficient, find

$$\beta_m = \arg \min_{\beta} \sum_{t=1}^T L(Y_t, f_{m-1}(X_t) + \beta h_m(X_t)). \quad (133)$$

Then iterating with some shrinkage/learning rate gives

$$f_m(X) = f_{m-1}(X) + \nu \beta_m h_m(X). \quad (134)$$

To speed up the process and include randomness, one can use stochastic gradient boosting, whereby a randomly selected subset of the data is chosen to optimize over at each step.

Example 5.1 (Gradient Boosting with MSE Loss). Consider predicting next-month returns for $n = 100$ stocks using 5 features. Initialize $f_0 = \bar{Y} = 0.8\%$. In round $m = 1$, the pseudo-residuals are $g_t^{(1)} = Y_t - 0.8\%$ for each stock-month. With learning rate $\nu = 0.1$, the update is $f_1(X) = 0.8\% + 0.1 \cdot h_1(X)$. After 500 rounds, the ensemble f_{500} captures complex nonlinear relationships between characteristics and returns that no single tree or linear model could represent.

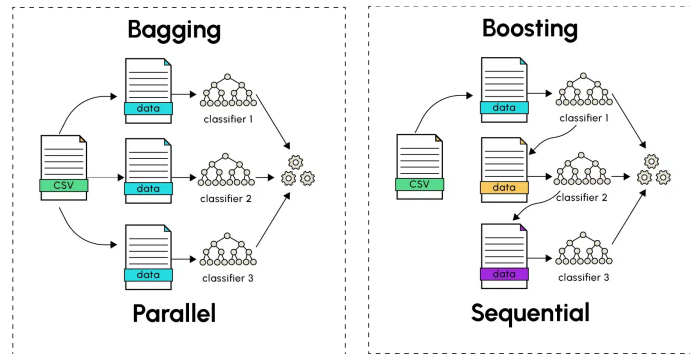


Figure 8: Bagging and boosting

Definition 5.5 (Bagging/Bootstrap Aggregating). For data $\mathcal{D}$ and desired predictor $\hat{f}(x; \mathcal{D})$, bootstrap B samples from $\mathcal{D}$ (sample with replacement to a certain size), then train separate models

$\hat{f}^{(b)}(x; \mathcal{D}^{(b)})$ on each bootstrapped dataset, and return an average:

$$\hat{f}^{(\text{bag})}(x; \mathcal{D}) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{(b)}(x; \mathcal{D}^{(b)}). \quad (135)$$

This is essentially Monte Carlo integrating the expectation of the estimator over the data distribution by bootstrap.

Proposition 5.1 (Variance Reduction by Bagging). *Suppose the B bootstrapped estimators $\hat{f}^{(b)}$ each have variance σ^2 and pairwise correlation $\rho \in [0, 1]$. Then the variance of the bagged estimator is*

$$\text{Var}(\hat{f}^{(\text{bag})}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2. \quad (136)$$

As $B \rightarrow \infty$, the second term vanishes and the variance floor is $\rho\sigma^2$. Bagging therefore reduces variance most when the individual trees are uncorrelated.

Proof. Write $\hat{f}^{(\text{bag})} = \frac{1}{B} \sum_{b=1}^B \hat{f}^{(b)}$. Then

$$\text{Var}(\hat{f}^{(\text{bag})}) = \frac{1}{B^2} \text{Var}\left(\sum_{b=1}^B \hat{f}^{(b)}\right) = \frac{1}{B^2} \left[\sum_{b=1}^B \text{Var}(\hat{f}^{(b)}) + \sum_{b \neq b'} \text{Cov}(\hat{f}^{(b)}, \hat{f}^{(b')}) \right]. \quad (137)$$

Since each term has variance σ^2 and each distinct pair has covariance $\rho\sigma^2$,

$$\text{Var}(\hat{f}^{(\text{bag})}) = \frac{1}{B^2} [B\sigma^2 + B(B-1)\rho\sigma^2] = \frac{\sigma^2}{B} + \frac{B-1}{B}\rho\sigma^2 = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2. \quad (138)$$

□

6 Tree-Based Methods

Regression trees are a flexible, non-parametric predictive model that recursively partition the input space via a greedy algorithm. Their main advantages are interpretability, the ability to handle categorical variables naturally, and robustness to monotone transformations of features. Their main disadvantage is instability: small perturbations to the data can lead to drastically different splits. Boosting and bagging resolve instability at the cost of interpretability.

Definition 6.1 (Regression Tree). Let $\vec{X}_t = (X_{t,1}, \dots, X_{t,p})^T$ and Y_t be such that

$$Y_t = \mathbb{E}[Y_t | \vec{X}_t] + U_t = f(\vec{X}_t) + U_t. \quad (139)$$

A regression tree approximates f by

$$H(\vec{X}_t) = \sum_{k=1}^K \beta_k I_k(\vec{X}_t), \quad I_k(\vec{X}_t) = \begin{cases} 1 & \text{if } \vec{X}_t \in R_k \\ 0 & \text{otherwise,} \end{cases} \quad (140)$$

where $R_1, \dots, R_K$ are disjoint regions (leaves) partitioning the input space. The approximation is equivalent to a linear regression on K indicator (dummy) variables. The prediction in each region is the sample average of Y_t over all observations falling in that region.

Example 6.1 (Regression Tree for Stock Returns). Consider predicting monthly stock returns R_t from two features: the price-to-earnings ratio PE_t and past 12-month momentum MOM_t . A depth-2 tree might produce the following partition:

1. **Split 1 (root):** $MOM_t < -10\%$

(a) Yes $\rightarrow$ predict $\hat{R} = -0.8\%$ (recent losers continue to fall in the short run).

(b) No $\rightarrow$ go to Split 2.

2. **Split 2:** $PE_t > 35$

(a) Yes $\rightarrow$ predict $\hat{R} = +0.2\%$ (expensive stocks with positive momentum).

(b) No $\rightarrow$ predict $\hat{R} = +1.1\%$ (reasonably valued stocks with positive momentum).

This tree captures a nonlinear interaction: momentum matters most on the downside, while valuation matters conditional on positive momentum. No linear model can represent this piecewise-constant structure.

Remark 6.1. More formally, a regression tree can be written as $Y_t = \sum_{i \in \mathcal{T}} \beta_i B_i(\vec{X}_t) + u_t$, where $\mathcal{T}$ is the set of terminal nodes (leaves) and $B_i(\vec{X}_t) = \mathbf{1}\{\vec{X}_t \in R_i\}$. Each region R_i is defined by a sequence of binary splits $X_{s_j} \leq c_j$ or $X_{s_j} > c_j$ along the path from the root to leaf i , so $B_i(\vec{X}_t) = \prod_{j \in J_i} \mathbf{1}\{\vec{X}_t \text{ follows split } j\}$, and $\sum_{i \in \mathcal{T}} B_i(\vec{X}_t) = 1$.

Definition 6.2 (Recursive Partitioning). Rather than solving the combinatorially intractable problem of finding the globally optimal partition, a regression tree is grown via a greedy algorithm with sequential binary splitting. At each step, given the current observations in a region R , find the split variable j and threshold c minimizing total within-child squared error by testing over all factors and thresholds (or a grid, in practice):

$$\min_{j, c} \left[\sum_{t: X_{t,j} < c} (Y_t - \hat{\beta}_{R_1})^2 + \sum_{t: X_{t,j} \geq c} (Y_t - \hat{\beta}_{R_2})^2 \right], \quad (141)$$

where $\hat{\beta}_{R_1}$ and $\hat{\beta}_{R_2}$ are the sample means of Y in the two child regions, which are the optimal constant predictions by the Best Estimator Theorem. Each split creates two new child regions:

$$R_1 = \{\vec{X} \mid X_j < c\}, \quad R_2 = \{\vec{X} \mid X_j \geq c\}. \quad (142)$$

Partitioning continues until the sum of squares within each region falls below a pre-specified value or the number of observations in each leaf reaches a minimum. Note that only axis-aligned splits are permitted; diagonal boundaries are not allowed.

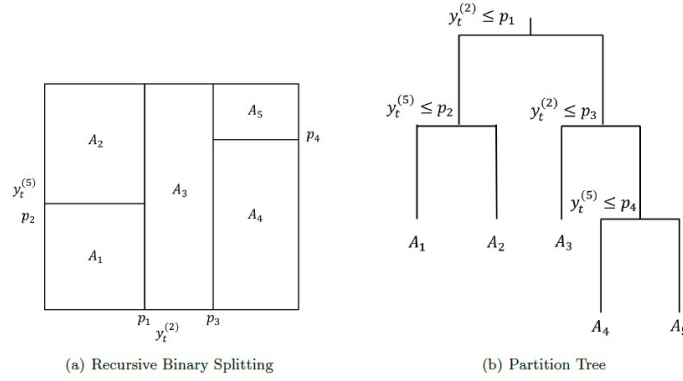


Figure 9: Tree partitioning

Remark 6.2 (Correctness and Limitations of Recursive Partitioning). The algorithm commits to each split permanently before recursing on the child regions. The globally optimal partition into K regions would require searching over all combinations of regions simultaneously, which is intractable. The greedy approximation has no optimality guarantee: a split that appears suboptimal locally might open up much better splits downstream, and vice versa. This is the fundamental source of instability: small perturbations to the data can change the best split at the root, which then propagates into an entirely different tree structure. Boosting and bagging address this by averaging over the instability rather than fixing it.

Definition 6.3 (Pruning). To avoid overfitting, a large tree T_0 is first grown, then pruned by solving a penalized loss problem. For each subtree $T \subseteq T_0$ with K leaves, define the cost-complexity criterion:

$$C_\alpha(T) = \sum_{k=1}^K Q_k(T) + \alpha K, \quad Q_k(T) = \sum_{\vec{X}_t \in R_k} (Y_t - \hat{\beta}_k)^2, \quad (143)$$

where $\alpha \geq 0$ is a tuning parameter selected by cross-validation. For a given α , there exists a unique smallest subtree $T_\alpha \subseteq T_0$ minimizing $C_\alpha(T)$. Large α penalizes complexity heavily and yields smaller trees; small α permits larger trees. The optimal T_α is found by *weakest-link pruning*: successively collapse the internal node that produces the smallest per-node increase in $\sum_k Q_k(T)$, generating a sequence of nested subtrees that contains T_α .

Definition 6.4 (Variable Importance). The relative *importance* of the i th predictor for a tree T is

$$I_i(T) = \sum_{j=1}^N \hat{\delta}_j^2 \cdot \mathbf{1}(s_j = i), \quad (144)$$

where N is the number of internal (parent) nodes, s_j is the index of the split variable at node j ,

and $\hat{\delta}_j^2$ is the reduction in residual sum of squares from split j :

$$\hat{\delta}_j^2 = \underbrace{\sum_{t \in R_j} (Y_t - \bar{Y}_j)^2}_{\text{RSS before split}} - \left[\underbrace{\sum_{t \in R_{j,L}} (Y_t - \bar{Y}_{j,L})^2}_{\text{left child}} + \underbrace{\sum_{t \in R_{j,R}} (Y_t - \bar{Y}_{j,R})^2}_{\text{right child}} \right]. \quad (145)$$

A variable is deemed important if it yields large reductions in prediction error across splits. For ensemble methods, the importance of variable i is averaged over all trees: $I_i = \frac{1}{M} \sum_{m=1}^M I_i(T_m)$.

Definition 6.5 (Partial Dependence Plot). For a fitted model $\hat{f}(\vec{X})$ and a subset of predictors $\vec{X}_S$ (with complement $\vec{X}_C$), the *partial dependence function* is

$$\hat{f}_S(\vec{x}_S) = \mathbb{E}_{\vec{X}_C} [\hat{f}(\vec{x}_S, \vec{X}_C)] \approx \frac{1}{T} \sum_{t=1}^T \hat{f}(\vec{x}_S, \vec{X}_{C,t}), \quad (146)$$

where the expectation is taken over the marginal distribution of $\vec{X}_C$, estimated empirically. A partial dependence plot traces $\hat{f}_S(\vec{x}_S)$ as $\vec{x}_S$ varies, displaying the average effect of $\vec{X}_S$ on the prediction after marginalizing out all other features. This provides interpretability for ensemble methods where the individual trees are not directly readable.

As previously discussed, gradient boosting builds an additive model sequentially, fitting each new weak learner to the pseudo-residuals of the current ensemble. When the weak learners are regression trees, the algorithm is as follows.

Definition 6.6 (Boosted Regression Tree Algorithm). Given a loss function L and learning rate $0 < \nu < 1$:

1. Initialize $f_0(\vec{X}) = \arg \min_c \sum_{t=1}^T L(Y_t, c)$.
2. For $m = 1, \dots, M$:
 - (a) Compute pseudo-residuals $r_{tm} = - \left[\frac{\partial L(Y_t, f(\vec{X}_t))}{\partial f(\vec{X}_t)} \right]_{f=f_{m-1}}$.
 - (b) Fit a regression tree to $\{r_{tm}\}$ with K_m leaves, yielding terminal regions R_{jm} .
 - (c) For each leaf j , compute the optimal step:

$$c_{jm} = \arg \min_c \sum_{\vec{X}_t \in R_{jm}} L[Y_t, f_{m-1}(\vec{X}_t) + c]. \quad (147)$$
 - (d) Update $f_m(\vec{X}) = f_{m-1}(\vec{X}) + \nu \sum_{j=1}^{K_m} c_{jm} \mathbf{1}(\vec{X} \in R_{jm})$.
3. Output $\hat{f}(\vec{X}) = f_M(\vec{X})$.

For squared error loss, $r_{tm} = Y_t - f_{m-1}(\vec{X}_t)$, so the tree is fit directly to the residuals.

Remark 6.3. Two tuning parameters govern the boosted tree: the tree depth K (controlling variable interaction) and the learning rate ν paired with the number of iterations M . In practice, small $\nu < 0.1$ with M chosen by early stopping is preferred. Shallow trees with $4 \leq K \leq 8$ are typical; a tree of depth $K = 2$ captures no variable interactions. To speed up training and introduce regularizing randomness, *stochastic gradient boosting* selects a random subsample of the data at each iteration.

Remark 6.4. A popular choice for robustness to outliers is the *Huber loss*:

$$L[Y_t, f(\vec{X}_t)] = \begin{cases} \frac{1}{2}[Y_t - f(\vec{X}_t)]^2 & \text{if } |Y_t - f(\vec{X}_t)| \leq \delta \\ \delta|Y_t - f(\vec{X}_t)| - \frac{1}{2}\delta^2 & \text{otherwise,} \end{cases} \quad (148)$$

where δ is taken as the α -quantile of the absolute residuals. The Huber loss behaves like squared error for small residuals but grows linearly for large ones, limiting the influence of outliers. The pseudo-residual is then

$$r_{tm} = \begin{cases} Y_t - f_{m-1}(\vec{X}_t) & \text{if } |Y_t - f_{m-1}(\vec{X}_t)| \leq \delta \\ \delta \cdot \text{sign}[Y_t - f_{m-1}(\vec{X}_t)] & \text{otherwise.} \end{cases} \quad (149)$$

Definition 6.7 (Random Forest). A *random forest* is a bagged ensemble of regression trees with an additional randomization over split variables, motivated by the variance reduction result of Proposition 5.1: since bagging cannot reduce variance below $\rho\sigma^2$, one must reduce ρ by decorrelating the trees. The algorithm is:

1. For $b = 1, \dots, B$:
 - (a) Draw a bootstrap sample of size T from the original data (with replacement).
 - (b) Fit a regression tree T_b to the bootstrap sample, but at each split, restrict the candidate split variables to a randomly chosen subset of $m \leq p$ variables. Grow the tree until a minimum leaf size is reached; do not prune.
2. Output the ensemble prediction:

$$\hat{f}^{(\text{RF})}(\vec{X}) = \frac{1}{B} \sum_{b=1}^B T_b(\vec{X}). \quad (150)$$

When $m = p$, this reduces to ordinary bagging. The canonical choice is $m = \lfloor \sqrt{p} \rfloor$.

Remark 6.5. The number of trees B can be monitored using the *out-of-bag* (OOB) error: for each observation $z_t = (Y_t, \vec{X}_t^T)^T$, the OOB prediction is the average of $T_b(\vec{X}_t)$ over only those bootstrap samples that did not include z_t . Since each observation is excluded from roughly one-third of bootstrap samples, this provides a nearly unbiased estimate of the generalization error without a separate validation set.

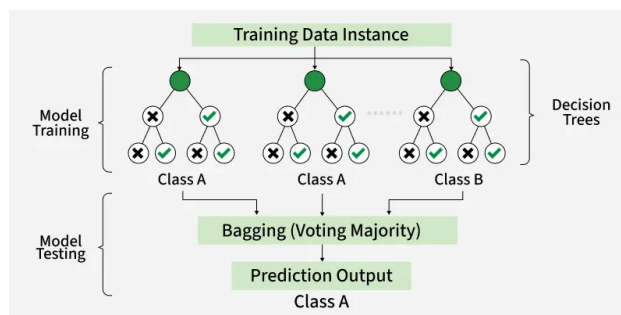


Figure 10: Random forest

Remark 6.6 (Tree-Based Methods in Finance). Gu, Kelly, and Xiu (2020) conduct a comprehensive comparison of machine learning methods for stock return prediction. Gradient-boosted trees and random forests achieve the highest OOS R^2 among all methods tested, outperforming both linear models and deep neural networks. The success of tree-based methods in finance is attributed to their ability to capture nonlinear interactions between firm characteristics, such as the interaction between momentum and volatility or between size and value, without requiring the researcher to specify these interactions in advance. Variable importance scores from random forests provide an empirical ranking of which characteristics matter most for return prediction, serving as a data-driven alternative to manually specifying factor models.

The bias-variance decomposition provides a unifying lens for all the estimators covered in the course. The following table summarizes the qualitative tradeoffs:

Method	Bias	Variance	Key mechanism
OLS	Low (none if model correct)	High in high dimensions	No regularization
Ridge	Introduced by λ	Reduced vs. OLS	Shrinks all $\hat{\beta}_j$ toward 0
LASSO	Introduced by λ	Reduced vs. OLS	Zeros out irrelevant $\hat{\beta}_j$
Deep tree	Low (flexible)	High (unstable)	Overfits training data
Pruned tree	Higher	Lower	Complexity penalty α
Random Forest	Low	Low	Averaging decorrelated trees
Boosting	Low	Low (with tuning)	Sequential bias reduction

Table 2: Qualitative bias-variance tradeoffs across regression methods.

7 Sieves

For $Y_t, \vec{X}_t$ having relationship $Y_t = h(\vec{X}_t) + U_t$, estimating the function $h(\cdot)$ over an infinite-dimensional function space $\mathcal{H}$,

$$\hat{h} = \arg \min_{h \in \mathcal{H}} \sum_{t=1}^T \left(Y_t - h(\vec{X}_t) \right)^2, \quad (151)$$

is infeasible because we do not have an efficient technique to search over all of $\mathcal{H}$.

Example 7.1 (Function Spaces). The function space $\mathcal{H}$ may take several forms:

- $\mathcal{C}$: the space of continuous functions on $[a, b]$.
- $\mathcal{C}^2$: the space of twice continuously differentiable functions.
- $\mathcal{B}$: the space of Borel measurable functions.

Of course, one can always restrict the space. For example, imposing linearity gives $\mathcal{H} = \mathcal{L} = \{\vec{\beta}' \vec{X}_t, \vec{\beta} \in \mathbb{R}^p\}$, the space of all linear functions of $\vec{X}_t$. This is the simplest case: $\mathcal{L}$ is finite-dimensional with dimension p . Beyond this simple case, the method of sieves offers a principled way to handle the general problem.

Definition 7.1 (Method of Sieves). Pick a sequence of finite-dimensional spaces $\{\mathcal{H}_D\}$, $D = 1, 2, 3, \dots$, that converges to $\mathcal{H}$. Define the approximating functions as

$$h^D(\vec{X}_t) = \sum_{j=1}^{J_T} \beta_j h_j(\vec{X}_t), \quad (152)$$

where $h_j(\cdot)$ is the j th basis function for $\mathcal{H}_D$ and can be either fully known (linear sieves) or indexed by a vector of parameters θ_j , such that $h_j(\vec{X}_t) = h(\vec{X}_t; \theta_j)$ (nonlinear sieves). Note that the number of basis functions $J := J_T$ depends on the sample size T . The *sieve estimator* is then

$$\hat{h}^D = \arg \min_{h \in \mathcal{H}_D} \sum_{t=1}^T \left(Y_t - h(\vec{X}_t) \right)^2. \quad (153)$$

The trick is to replace the intractable $\mathcal{H}$ by a simpler finite-dimensional space $\mathcal{H}_D$.

Remark 7.1. The dimension D of the space $\mathcal{H}_D$ also depends on the sample size: $D := D_T$. When the basis functions have no free parameters, $D = J$; when they are indexed by parameters θ , $D > J$, as the parameter count exceeds the number of basis functions. Popular sieves are typically compact, non-decreasing ($\mathcal{H}_T \subseteq \mathcal{H}_{T+1} \subseteq \dots \subseteq \mathcal{H}$), and are constructed so that for any $h \in \mathcal{H}$, there exists a sequence $\pi_T h \in \mathcal{H}_T$ satisfying $d(h, \pi_T h) \rightarrow 0$ as $T \rightarrow \infty$, where π_T is a projection map from $\mathcal{H}$ into $\mathcal{H}_T$.

The construction of the approximating sequence $\{\mathcal{H}_D\}$ relies on the concept of dense sets.

Definition 7.2 (Dense Set). If A and B are subsets of a metric space $\mathcal{X}$, then A is *dense* in B if for any $\varepsilon > 0$ and any $X \in B$, there exists $Y \in A$ such that $\|X - Y\|_{\mathcal{X}} < \varepsilon$.

Theorem 7.1 (Stone–Weierstrass). *Let $\mathcal{H} = \mathcal{C}$ = space of continuous functions on $[a, b]$. From the theory of approximating functions, the proper subset $\mathcal{P} \subset \mathcal{C}$ of polynomials is dense in $\mathcal{C}$. The key result is the Stone–Weierstrass theorem: for any continuous function $m(\cdot)$ with compact support $\mathcal{X}$ on $\mathbb{R}$, and any $\varepsilon > 0$, there exists a polynomial function $P(X)$ such that $\sup_{X \in \mathcal{X}} |m(X) - P(X)| < \varepsilon$. It is therefore natural to define the sequence of approximating spaces $\mathcal{H}_D$, $D = 1, 2, 3, \dots$ by taking $\mathcal{H}_D$ to be the set of polynomials of degree at most $D - 1$ (including a constant term), so that $\dim(\mathcal{H}_D) = D < \infty$. In the limit, this sequence converges to the infinite-dimensional space of polynomials, which is dense in $\mathcal{C}$.*

When the basis functions are fully known and the approximating function is linear in the parameters, estimation reduces to familiar linear regression methods.

Definition 7.3 (Polynomial Sieve). For $X \in \mathbb{R}$, let $\text{Pol}(J_T)$ denote the space of polynomials on $[0, 1]$ of degree at most J_T :

$$\text{Pol}(J_T) = \left\{ \sum_{j=0}^{J_T} \beta_j X^j, X \in [0, 1] : \beta_j \in \mathbb{R} \right\}. \quad (154)$$

For $p = 1$, the *polynomial sieve* approximates $h(X_t)$ by

$$h(X_t) = \beta_0 + \beta_1 X_t + \beta_2 X_t^2 + \dots + \beta_J X_t^J. \quad (155)$$

For $X \in \mathbb{R}^p$ with $p > 1$, the polynomial sieve is generalized by including cross terms. For $p = 2$:

$$h(\vec{X}_t) = \beta_0 + \beta_1 X_1 + \beta_2 X_1^2 + \beta_3 X_2 + \beta_4 X_2^2 + \beta_5 X_1 X_2 + \dots \quad (156)$$

For general $p > 1$, the polynomial sieve of order J_T includes all monomials of total degree at most J_T :

$$h(\vec{X}_t) = \beta_0 + \sum_{i_1=1}^p \beta_{i_1} X_{i_1} + \sum_{i_1=1}^p \sum_{i_2=i_1}^p \beta_{i_1 i_2} X_{i_1} X_{i_2} + \dots + \sum_{i_1=1}^p \dots \sum_{i_{J_T}=i_{J_T-1}}^p \beta_{i_1 \dots i_{J_T}} X_{i_1} \times \dots \times X_{i_{J_T}}. \quad (157)$$

An alternative orthogonal basis is given by Legendre polynomials, which are orthogonal on $[-1, 1]$: the j th Legendre polynomial is $p_j(\tau) = \frac{1}{2^j j!} \frac{d^j}{d\tau^j} [(x^2 - 1)^j]$, with $p_1 = 1$, $p_2 = \tau$, $p_3 = \frac{3\tau^2 - 1}{2}$, $p_4 = \frac{5\tau^3 - 3\tau}{2}$. To model a function on $[0, T_{\max}]$, one applies the change of variable $\tau \mapsto \frac{2\tau}{T_{\max}} - 1$.

Proposition 7.1 (Estimation of Linear Sieves). *When the basis functions are fully known (linear sieves), the problem is linear in the parameters. If $J \ll T$, the parameter vector $\vec{\beta} = (\beta_1, \dots, \beta_J)'$ is estimated by OLS:*

$$\hat{\vec{\beta}} = (X_J' X_J)^{-1} X_J' \vec{Y}, \quad (158)$$

where X_J is the $T \times (J + 1)$ design matrix with (t, j) -entry $h_j(X_t)$, and $\vec{Y} = (Y_1, \dots, Y_T)'$. If J is

large relative to T , penalized estimators such as Ridge or LASSO can be used. The number of basis functions J is selected by cross-validation or information criteria (AIC, BIC).

Remark 7.2 (Curse of Dimensionality). For $p > 1$, the number of monomials of total degree at most J grows as $\binom{p+J}{J} \sim p^J/J!$, which can become very large. In $p = 10$ dimensions with $J = 3$, there are already $\binom{13}{3} = 286$ basis functions, making LASSO regularization or orthogonal bases essential for feasible estimation.

Example 7.2 (Polynomial Sieve Fit). Consider the data-generating process $y = 0.2 - 10x + x \exp(x^2) + \varepsilon$ with $\varepsilon \sim N(0, 100)$. A linear fit ($J = 1$) captures only the dominant slope and misses the non-linearity. A 3rd-order polynomial begins to recover the shape, while a 5th-order fit tracks the true curve more closely. By the 10th order, the fitted polynomial is nearly indistinguishable from the true function over $[-2, 2]$.

When the basis functions are themselves indexed by parameters θ_j , the problem is nonlinear in the full parameter vector and requires nonlinear least squares.

Definition 7.4 (Sigmoid Neural Network Sieve). The *single hidden layer sigmoid artificial neural network (sANN) sieve* of width J_T is defined as

$$\text{sANN}(J_T) = \left\{ \gamma_0 + \sum_{j=1}^{J_T} \alpha_j S(\vec{\gamma}'_j \vec{X} + \gamma_{0,j}) : \vec{\gamma}_j \in \mathbb{R}^p, \alpha_j, \gamma_0, \gamma_{0,j} \in \mathbb{R} \right\}, \quad (159)$$

where $S : \mathbb{R} \rightarrow \mathbb{R}$ is a sigmoid activation function, i.e., a bounded nondecreasing function such that $\lim_{u \rightarrow -\infty} S(u) = 0$ and $\lim_{u \rightarrow +\infty} S(u) = 1$. The canonical example is the logistic function $S(x) = 1/(1 + e^{-x})$.

Remark 7.3 (Neural Networks as Nonlinear Sieves). In the sANN sieve, the parameters $\vec{\gamma}_j$ and $\gamma_{0,j}$ index the basis functions themselves, making the problem nonlinear: $D > J_T$. As $J_T \rightarrow \infty$, the sANN sieve fills out the full space of continuous functions by the Universal Approximation Theorem. The dimension J_T plays the same role as the polynomial degree D : it controls the expressiveness of the approximating space and must grow with T at an appropriate rate to balance bias and variance. For nonlinear sieves, information criteria are less tractable (degrees of freedom are difficult to determine), making regularization via weight decay or Bayesian priors the preferred approach for controlling overfitting.

Example 7.3 (Neural Network Sieve Fit). Applied to the same data-generating process $y = 0.2 - 10x + x \exp(x^2) + \varepsilon$, a shallow sigmoid network with $m = 1$ neuron recovers only a monotone transformation of x , performing similarly to the linear fit. With $m = 3$ neurons, the network captures the S-shaped structure of the true function. At $m = 7$ and $m = 10$ neurons with Bayesian regularization, the fitted curve closely approximates the true function throughout $[-2, 2]$. This mirrors the behavior of polynomial sieves: the fit improves as J_T grows, but regularization is essential to prevent overfitting, especially near the boundaries.

The method of sieves generalizes naturally to semi-parametric models, which combine a finite-dimensional parametric component with an infinite-dimensional nonparametric component.

Definition 7.5 (Semi-Parametric Sieve Model). Consider the general *semi-parametric model*

$$Y_t = \underbrace{\vec{\beta}_0' \vec{X}_t}_{\text{parametric}} + \underbrace{h_0(\vec{X}_t)}_{\text{nonparametric}} + U_t = m_0(\vec{X}_t; \theta_0) + U_t, \quad (160)$$

where $\{U_t\}$ is a sequence of disturbances with $\mathbb{E}[U_t | \vec{X}_t] = 0$ and $\mathbb{E}[U_t^2 | \vec{X}_t] = \sigma^2(\vec{X}_t) < \infty$. The full parameter is $\theta_0 = (\vec{\beta}_0', h_0)' \in \mathcal{B} \times \mathcal{H} \equiv \Theta$, where $\mathcal{B}$ is a finite-dimensional compact parameter space and $\mathcal{H}$ is infinite-dimensional. The population criterion is uniquely minimized at θ_0 :

$$\theta_0 = \arg \min_{\theta \in \Theta} \mathbb{E} \left[\left(Y - m(\vec{X}; \theta) \right)^2 \right], \quad (161)$$

and the sieve estimator replaces Θ with an approximating sequence Θ_T :

$$\hat{\theta} = \arg \min_{\theta \in \Theta_T} \sum_{t=1}^T \left(Y_t - m(\vec{X}_t; \theta) \right)^2. \quad (162)$$

A particularly important application of sieve methods in finance is the estimation of the term structure of interest rates.

Example 7.4 (Term Structure Estimation). The term structure of interest rates is a continuous curve mapping time to maturity τ into the interest rate $R(t, \tau)$ used to discount cash flows at maturity τ . Since the curve must be estimated at all maturities in a compact interval, sieve methods are natural. Given observed yields $\{R(t, \tau_i)\}_{i=1, \dots, N}$, the term structure is approximated by a linear combination of basis functions $\{f_j(\tau | \theta_t)\}_{j=1, 2, \dots}$:

$$\hat{R}(t, \tau, \vec{\beta}_t, \theta_t) = \sum_{j=1}^K \beta_{t,j} f_j(\tau | \theta_t). \quad (163)$$

The parameters $\{\theta_t, \vec{\beta}_t\}$ are estimated by least squares:

$$\min_{\{\theta_t, \vec{\beta}_t\}} \sum_{i=1}^N \left(R(t, \tau_i) - \hat{R}(t, \tau_i, \vec{\beta}_t, \theta_t) \right)^2, \quad (164)$$

where each $f_j(\tau | \theta_t)$ represents a movement of the term structure and $\beta_{t,j}$ is the magnitude of that movement at time t . The number of factors K is typically chosen by PCA or information criteria.

Example 7.5 (Power Polynomial and Legendre Sieves). Two canonical linear sieves are used in term structure modeling:

Power polynomials. The basis $\{f_j(\tau) = \tau^{j-1}\}_{j=1, 2, \dots}$ gives

$$\hat{R}(t, \tau, \vec{\beta}_t) = \beta_{t,1} + \beta_{t,2}\tau + \beta_{t,3}\tau^2 + \beta_{t,4}\tau^3 + \dots, \quad \tau \geq 0. \quad (165)$$

The first four basis functions have clear interpretations: level (τ^0), slope (τ^1), curvature (τ^2), and double curvature (τ^3). These functions are linearly independent but not orthogonal, so estimation by OLS may be numerically sensitive.

Legendre polynomials. The alternative orthogonal basis replaces power monomials with Legendre polynomials $\{p_j(\tau)\}$, which satisfy $\int_{-1}^1 p_i(\tau)p_j(\tau) d\tau = 0$ for $i \neq j$. After the change of variable $\tau \mapsto 2\tau/T_{\max} - 1$ mapping $[0, T_{\max}]$ to $[-1, 1]$, estimation reduces to OLS with an orthogonal design matrix, yielding more stable coefficient estimates. The first four Legendre components carry the same economic interpretations as in the power case.

Almeida et al. (1998) adopted Legendre polynomial sieves to extract the term structure in emerging markets where the curve is not directly observable from zero-coupon bonds but must be inferred from coupon-bearing instruments. Almeida et al. (2017) extended this framework with dynamic exponential splines to forecast U.S. zero-coupon yields out-of-sample, allowing for segmentation between the short and long ends of the term structure.

8 Clustering

Unlike supervised learning, clustering is an unsupervised problem: there is no response variable y , and the goal is to discover latent structure in the feature data X alone. The central question is how to partition observations into groups such that members of the same group are more similar to one another than to members of other groups. Because there is no ground truth to evaluate against, model selection and validation are more challenging than in the supervised setting.

Definition 8.1 (Dissimilarity Measure). Let $\vec{x}_i, \vec{x}_j \in \mathbb{R}^p$. A *dissimilarity measure* $d(\vec{x}_i, \vec{x}_j) \geq 0$ quantifies how unlike two observations are, satisfying $d(\vec{x}_i, \vec{x}_i) = 0$ and $d(\vec{x}_i, \vec{x}_j) = d(\vec{x}_j, \vec{x}_i)$ (metric). The most common choice is squared Euclidean distance:

$$d(\vec{x}_i, \vec{x}_j) = \|\vec{x}_i - \vec{x}_j\|_2^2 = \sum_{k=1}^p (x_{ik} - x_{jk})^2. \quad (166)$$

Features should be standardized to zero mean and unit variance before computing dissimilarities, as variables measured on larger scales will otherwise dominate.

Definition 8.2 (K -Means Clustering). Given observations $\{\vec{x}_i\}_{i=1}^n$ and a number of clusters K , *K-means clustering* seeks a partition $C : \{1, \dots, n\} \rightarrow \{1, \dots, K\}$ minimizing the within-cluster sum of squares (WCSS):

$$\min_{C, \{\vec{\mu}_k\}_{k=1}^K} \sum_{k=1}^K \sum_{C(i)=k} \|\vec{x}_i - \vec{\mu}_k\|_2^2, \quad (167)$$

where $\vec{\mu}_k = \frac{1}{|C_k|} \sum_{C(i)=k} \vec{x}_i$ is the centroid of cluster k . This is solved via *Lloyd's algorithm*:

1. Initialize K centroids $\vec{\mu}_1, \dots, \vec{\mu}_K$ (e.g., by selecting K observations at random).
2. Assign each observation to its nearest centroid: $C(i) = \arg \min_k \|\vec{x}_i - \vec{\mu}_k\|_2^2$.
3. Recompute centroids as the mean of their assigned observations.
4. Repeat steps 2–3 until assignments do not change.

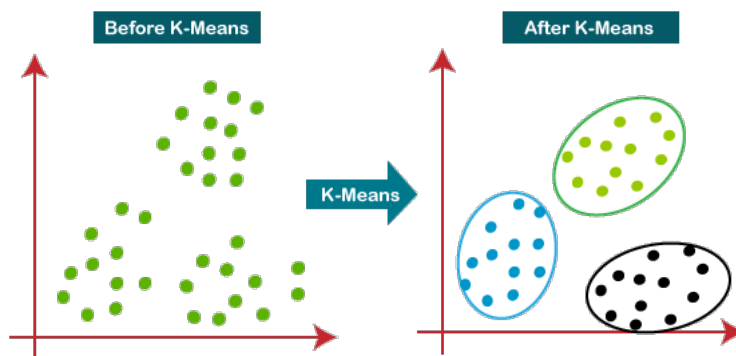


Figure 11: K-means clustering

Proposition 8.1 (Convergence of K -Means). *Lloyd's algorithm decreases the WCSS objective at every step and therefore converges in a finite number of iterations.*

Proof. The assignment step minimizes the objective over C with centroids held fixed: assigning each observation to its nearest centroid cannot increase the total squared distance. The update step minimizes the objective over $\{\vec{\mu}_k\}$ with C held fixed: for fixed C , $\sum_{C(i)=k} \|\vec{x}_i - \vec{\mu}_k\|_2^2$ is a sum of squared deviations minimized by $\vec{\mu}_k = \frac{1}{|C_k|} \sum_{C(i)=k} \vec{x}_i$. Since the objective is non-negative and strictly decreases at each step unless already at a fixed point, and there are finitely many partitions, convergence is guaranteed. $\square$

Remark 8.1. K -means converges to a local, not necessarily global, minimum. In practice, the algorithm is run many times with different random initializations, and the solution with the lowest WCSS is retained. A smarter initialization, K -means++, selects the first centroid uniformly at random, then selects each subsequent centroid with probability proportional to its squared distance from the nearest already-chosen centroid. This tends to produce better solutions and faster convergence.

Remark 8.2. Choosing K is a model selection problem. A common heuristic is the *elbow method*: plot the WCSS as a function of K and look for the point of diminishing returns. A more principled alternative is the *gap statistic*, which compares the observed WCSS to its expectation under a null reference distribution with no cluster structure, selecting the K that maximizes the gap.

Example 8.1 (Clustering Stocks by Return Characteristics). An asset manager clusters $n = 2000$ U.S. stocks using $p = 6$ standardized features: market beta, size (log market cap), book-to-market, momentum, profitability, and investment rate. Running K -means with $K = 5$ and 100 random restarts yields the following cluster centroids (illustrative):

Cluster	β	Size	B/M	Mom	Prof	Inv
1 (“Large Growth”)	1.05	+1.8	−0.9	+0.3	+0.6	+0.4
2 (“Small Value”)	0.95	−1.2	+1.3	−0.1	−0.4	−0.6
3 (“High Momentum”)	1.20	+0.2	−0.2	+1.8	+0.3	+0.1
4 (“Defensive”)	0.55	+0.8	+0.4	+0.1	+0.9	−0.3
5 (“Distressed”)	1.40	−1.5	+0.8	−1.6	−1.2	−0.2

These clusters correspond to economically meaningful “styles.” Monitoring the relative sizes of clusters over time can reveal regime shifts; for example, the “Distressed” cluster might grow during recessions.

Definition 8.3 (Hierarchical Clustering). *Hierarchical clustering* produces a nested sequence of partitions, represented as a *dendrogram*, without requiring K to be specified in advance. The agglomerative (bottom-up) approach begins with each observation in its own cluster and successively merges the two closest clusters until all observations form a single group. At each step, the distance between two clusters G and H is computed via a *linkage function*:

$$d_{\text{single}}(G, H) = \min_{i \in G, j \in H} d(\vec{x}_i, \vec{x}_j), \quad (168)$$

$$d_{\text{complete}}(G, H) = \max_{i \in G, j \in H} d(\vec{x}_i, \vec{x}_j), \quad (169)$$

$$d_{\text{average}}(G, H) = \frac{1}{|G||H|} \sum_{i \in G} \sum_{j \in H} d(\vec{x}_i, \vec{x}_j). \quad (170)$$

A partition into K clusters is obtained by cutting the dendrogram at the appropriate height.

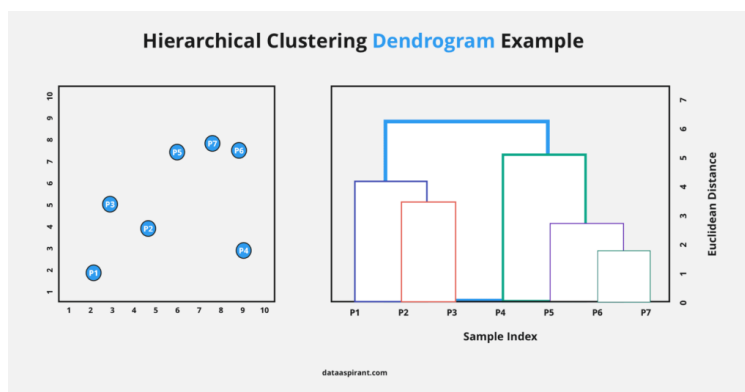


Figure 12: Dendrogram

Remark 8.3. Complete linkage tends to produce compact, roughly equal-sized clusters and is less susceptible to chaining (where a single outlier causes two large clusters to merge). Single linkage can identify elongated clusters but is sensitive to outliers. Average linkage is a compromise. The choice of linkage is substantive and should be guided by the application.

Definition 8.4 (Gaussian Mixture Model). A *Gaussian mixture model (GMM)* assumes observations are drawn from a mixture of K Gaussian distributions:

$$p(\vec{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\vec{x}; \vec{\mu}_k, \Sigma_k), \quad (171)$$

where $\pi_k > 0$ are the mixing proportions satisfying $\sum_k \pi_k = 1$, and $\mathcal{N}(\vec{x}; \vec{\mu}_k, \Sigma_k)$ is the multivariate Gaussian density with mean $\vec{\mu}_k$ and covariance Σ_k . The parameters $\theta = \{\pi_k, \vec{\mu}_k, \Sigma_k\}_{k=1}^K$ are estimated by maximum likelihood.

Definition 8.5 (Expectation-Maximization Algorithm). Since the component assignments are unobserved, direct maximization of the GMM log-likelihood is intractable. The *EM algorithm* maximizes the expected complete-data log-likelihood by iterating two steps. Let $r_{ik} = \mathbb{P}(Z_i = k \mid \vec{x}_i, \theta)$ be the *responsibility* of component k for observation i :

1. E-step: Compute responsibilities given current parameters:

$$r_{ik} = \frac{\pi_k \mathcal{N}(\vec{x}_i; \vec{\mu}_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\vec{x}_i; \vec{\mu}_j, \Sigma_j)}. \quad (172)$$

2. M-step: Update parameter estimates using the weighted sufficient statistics. Let $n_k = \sum_i r_{ik}$ and define the update with the following prior class probability, mean, and covariance:

$$\hat{\pi}_k = \frac{n_k}{n}, \quad \hat{\vec{\mu}}_k = \frac{1}{n_k} \sum_i r_{ik} \vec{x}_i, \quad \hat{\Sigma}_k = \frac{1}{n_k} \sum_i r_{ik} (\vec{x}_i - \hat{\vec{\mu}}_k)(\vec{x}_i - \hat{\vec{\mu}}_k)^T. \quad (173)$$

The EM algorithm is guaranteed to increase the observed-data log-likelihood at each iteration.

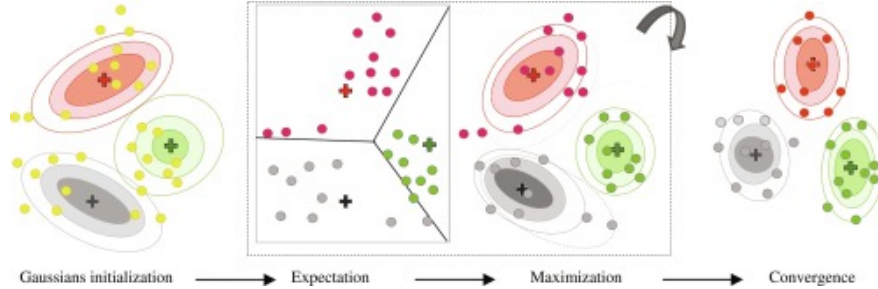


Figure 13: EM algorithm

Proposition 8.2 (EM Monotonically Increases Log-Likelihood). Let $\theta^{(t)}$ and $\theta^{(t+1)}$ be the parameters before and after one EM iteration. Then $\ell(\theta^{(t+1)}) \geq \ell(\theta^{(t)})$, where $\ell(\theta) = \sum_i \log p(\vec{x}_i; \theta)$ is the observed-data log-likelihood.

Proof sketch. Define the complete-data log-likelihood $\ell_c(\theta) = \sum_i \log p(\vec{x}_i, Z_i; \theta)$ and the auxiliary function $Q(\theta|\theta^{(t)}) = \mathbb{E}_{Z|X, \theta^{(t)}}[\ell_c(\theta)]$. The E-step computes $Q(\theta|\theta^{(t)})$, and the M-step chooses

$\theta^{(t+1)} = \arg \max_{\theta} Q(\theta|\theta^{(t)})$. By Jensen's inequality applied to the KL divergence between the posterior $p(Z|X, \theta^{(t)})$ and $p(Z|X, \theta)$, it can be shown that $\ell(\theta) \geq Q(\theta|\theta^{(t)}) + H(\theta^{(t)})$ for an entropy term H that does not depend on θ . Since the M-step ensures $Q(\theta^{(t+1)}|\theta^{(t)}) \geq Q(\theta^{(t)}|\theta^{(t)})$, the observed log-likelihood cannot decrease. $\square$

Remark 8.4. K -means is a special case of EM for GMMs where all covariance matrices are constrained to $\Sigma_k = \sigma^2 I$ and responsibilities are hard (i.e., each observation is assigned to exactly one component). GMMs provide a probabilistic, soft-assignment generalization, and their log-likelihood can be used with BIC for model selection over K .

Remark 8.5 (Clustering in Finance). GMMs are used in finance for *regime detection*: fitting a 2- or 3-component mixture to market returns identifies “bull,” “bear,” and “crisis” regimes with distinct mean returns and volatilities. The posterior probabilities r_{ik} give a real-time estimate of the current regime, which can be used to adjust portfolio allocations (e.g., reducing equity exposure when the “crisis” component has high responsibility). Hierarchical clustering of the correlation matrix of assets is used for portfolio diversification: assets within the same cluster are substitutes, so a well-diversified portfolio should hold assets from different clusters. This “hierarchical risk parity” approach has gained traction as an alternative to mean-variance optimization.

9 Classification

Classification is the supervised learning problem where the response Y takes values in a finite set of classes $\{1, \dots, K\}$. Logistic regression (Lecture 2) provides one approach for $K = 2$.

Definition 9.1 (Bayes Classifier). The *Bayes classifier* assigns each observation $\vec{x}$ to the class with highest posterior probability:

$$\hat{C}(\vec{x}) = \arg \max_k \mathbb{P}(Y = k | \vec{X} = \vec{x}). \quad (174)$$

By Bayes' theorem, $\mathbb{P}(Y = k|\vec{x}) \propto \mathbb{P}(Y = k)\mathbb{P}(\vec{x}|Y = k) = \pi_k f_k(\vec{x})$, where $\pi_k = \mathbb{P}(Y = k)$ is the prior class probability and $f_k(\vec{x})$ is the class-conditional density $\mathbb{P}(\vec{x}|Y = k)$. The Bayes classifier achieves the lowest possible misclassification rate (the Bayes error rate), but is infeasible in practice since f_k is unknown.

Proposition 9.1 (Optimality of the Bayes Classifier). *The Bayes classifier minimizes the misclassification probability among all classifiers.*

Proof. Let $C^*(\vec{x}) = \arg \max_k \mathbb{P}(Y = k|\vec{x})$ be the Bayes classifier and $\hat{C}$ any other classifier. The misclassification probability of $\hat{C}$ is

$$\mathbb{P}(\hat{C}(\vec{X}) \neq Y) = \mathbb{E} \left[\mathbb{P}(\hat{C}(\vec{X}) \neq Y | \vec{X}) \right] = \mathbb{E} \left[1 - \mathbb{P}(Y = \hat{C}(\vec{X}) | \vec{X}) \right]. \quad (175)$$

Since C^* maximizes $\mathbb{P}(Y = k|\vec{X})$ pointwise, $\mathbb{P}(Y = C^*(\vec{X})|\vec{X}) \geq \mathbb{P}(Y = \hat{C}(\vec{X})|\vec{X})$ for all $\vec{X}$, so $\mathbb{P}(C^*(\vec{X}) \neq Y) \leq \mathbb{P}(\hat{C}(\vec{X}) \neq Y)$. $\square$

Definition 9.2 (Linear Discriminant Analysis). *Linear discriminant analysis (LDA)* estimates the Bayes classifier by modeling each class-conditional density as Gaussian with a shared covariance matrix:

$$\vec{X} \mid Y = k \sim \mathcal{N}(\vec{\mu}_k, \Sigma). \quad (176)$$

Parameters are estimated by MLE: $\hat{\pi}_k = n_k/n$, $\hat{\vec{\mu}}_k = \frac{1}{n_k} \sum_{C(i)=k} \vec{x}_i$, and $\hat{\Sigma} = \frac{1}{n-K} \sum_k \sum_{C(i)=k} (\vec{x}_i - \hat{\vec{\mu}}_k)(\vec{x}_i - \hat{\vec{\mu}}_k)^T$. Then, examine the log-posteriors from the Bayesian $\mathbb{P}(Y = k|\vec{x}) \propto \pi_k f_k(\vec{x})$:

$$\ln \left(\pi_k \frac{1}{(2\pi)^{p/2} |\Sigma_k|^{1/2}} e^{-\frac{1}{2}(\vec{x} - \vec{\mu}_k)^T \Sigma_k^{-1} (\vec{x} - \vec{\mu}_k)} \right) = \ln \pi_k - \ln \left((2\pi)^{p/2} |\Sigma_k|^{1/2} \right) - \frac{1}{2}(\vec{x} - \vec{\mu}_k)^T \Sigma_k^{-1} (\vec{x} - \vec{\mu}_k). \quad (177)$$

The Bayesian classifier assigns the data point to the class with the highest posterior probability, so neglect terms that are identical across classes. Since the covariance matrix is shared, $\Sigma_k = \Sigma$. The LDA decision rule thus assigns $\vec{x}$ to the class k maximizing the *linear discriminant function*:

$$\delta_k(\vec{x}) = \vec{x}^T \hat{\Sigma}^{-1} \hat{\vec{\mu}}_k - \frac{1}{2} \hat{\vec{\mu}}_k^T \hat{\Sigma}^{-1} \hat{\vec{\mu}}_k + \log \hat{\pi}_k. \quad (178)$$

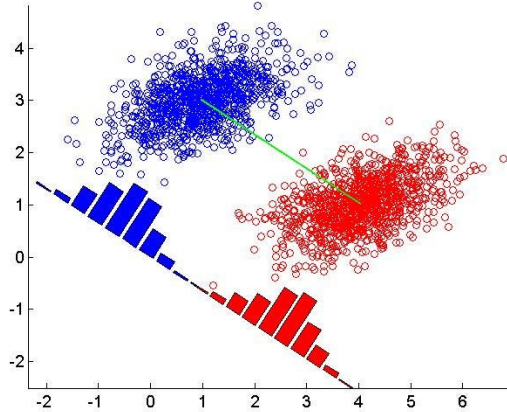


Figure 14: Linear discriminant analysis, showing projected Gaussians with the same variance

Proposition 9.2 (LDA Decision Boundary). *The LDA decision boundary between classes k and l is linear in $\vec{x}$.*

Proof. The boundary is the set of $\vec{x}$ where $\delta_k(\vec{x}) = \delta_l(\vec{x})$, i.e.,

$$\vec{x}^T \hat{\Sigma}^{-1} (\hat{\vec{\mu}}_k - \hat{\vec{\mu}}_l) = \frac{1}{2} (\hat{\vec{\mu}}_k^T \hat{\Sigma}^{-1} \hat{\vec{\mu}}_k - \hat{\vec{\mu}}_l^T \hat{\Sigma}^{-1} \hat{\vec{\mu}}_l) - \log \frac{\hat{\pi}_k}{\hat{\pi}_l}. \quad (179)$$

The left side is linear in $\vec{x}$ and the right side is a scalar constant, so this defines a hyperplane. $\square$

Example 9.1 (LDA for Market Regime Classification). An analyst classifies monthly market condi-

tions as “bull” ($Y = 1$) or “bear” ($Y = 0$) using two features: the trailing 3-month market return (x_1) and the VIX level (x_2). From historical data: $\hat{\mu}_1 = (3.5, 15)^T$ (bull), $\hat{\mu}_0 = (-2.1, 28)^T$ (bear), $\hat{\pi}_1 = 0.7$, $\hat{\pi}_0 = 0.3$. The shared covariance is $\hat{\Sigma} = \begin{pmatrix} 12 & -8 \\ -8 & 25 \end{pmatrix}$. For a new month with $\vec{x} = (1.0, 20)^T$, computing $\delta_1(\vec{x})$ and $\delta_0(\vec{x})$ and comparing them yields the classification. The decision boundary $\delta_1(\vec{x}) = \delta_0(\vec{x})$ is a line in the (return, VIX) plane: roughly, months with high returns and low VIX are classified as bull markets.

Definition 9.3 (Quadratic Discriminant Analysis). *Quadratic discriminant analysis (QDA)* relaxes the shared covariance assumption of LDA, allowing each class its own covariance Σ_k :

$$\vec{X} \mid Y = k \sim \mathcal{N}(\vec{\mu}_k, \Sigma_k). \quad (180)$$

The Bayesian classifier method of assigning the data point to the class with the highest posterior probability still holds, but now terms including Σ_k cannot be neglected. The decision rule hence assigns $\vec{x}$ to the class maximizing:

$$\delta_k(\vec{x}) = -\frac{1}{2} \log |\Sigma_k| - \frac{1}{2} (\vec{x} - \vec{\mu}_k)^T \Sigma_k^{-1} (\vec{x} - \vec{\mu}_k) + \log \pi_k. \quad (181)$$

The decision boundary between any two classes is now quadratic in $\vec{x}$. QDA requires estimating K separate covariance matrices, which is more flexible but has higher variance than LDA.

Remark 9.1. LDA and QDA represent a bias-variance tradeoff: LDA imposes a strong constraint (shared Σ) that reduces variance at the cost of potential bias if class covariances differ substantially. LDA is preferred when n is small relative to p ; QDA is preferred when class covariances differ and n is large enough to estimate them reliably.

Definition 9.4 (Confusion Matrix and Performance Metrics). For a binary classifier with $Y \in \{0, 1\}$, the *confusion matrix* tabulates the four possible outcomes:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad (182)$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad (183)$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \quad (184)$$

where TP, TN, FP, FN denote true positives, true negatives, false positives, and false negatives respectively. The *receiver operating characteristic* (ROC) curve plots the true positive rate (Recall) against the false positive rate $FP/(FP + TN)$ as the classification threshold parametrically varies. The *area under the ROC curve* (AUC) summarizes classifier performance across all thresholds, with $\text{AUC} = 1$ indicating a perfect classifier and $\text{AUC} = 0.5$ indicating random guessing.

Remark 9.2. Accuracy is a misleading metric when class frequencies are imbalanced; a classifier that predicts the majority class for every observation can achieve high accuracy while completely failing to identify the minority class. Precision, recall, F_1 , and AUC are more informative in such settings, and the choice among them depends on the relative costs of false positives and false negatives in the application.

The classification methods covered so far model the conditional probability $\mathbb{P}(Y = 1|\vec{X})$ explicitly. Support vector methods take a different route: they search directly for the decision boundary that best separates the classes, without committing to a full probabilistic model. The resulting estimators are robust in high-dimensional settings where $d \gg n$.

Definition 9.5 (Linear Classifier). Let $Y \in \{-1, +1\}$ and $\vec{X} \in \mathbb{R}^d$. A *linear classifier* takes the form

$$f(\vec{X}; \vec{w}, b) = \text{sgn}(\vec{w} \cdot \vec{X} + b), \quad (185)$$

where $\text{sgn}(z) = +1$ if $z \geq 0$ and -1 otherwise, $\vec{w} \in \mathbb{R}^d$ is the normal vector to the decision hyperplane, and $b \in \mathbb{R}$ is the offset. The hyperplane $\{\vec{x} : \vec{w} \cdot \vec{x} + b = 0\}$ partitions $\mathbb{R}^d$ into a positive half-space ($\vec{w} \cdot \vec{x} + b > 0$) and a negative half-space ($\vec{w} \cdot \vec{x} + b < 0$). The classifier assigns $\hat{Y} = +1$ to observations on the positive side and $\hat{Y} = -1$ to those on the negative side.

Definition 9.6 (Linear Separability and Support Vectors). A training set $\{(\vec{X}_i, Y_i)\}_{i=1}^n$ is *linearly separable* if there exists $(\vec{w}, b)$ such that $\vec{w} \cdot \vec{X}_i + b > 0$ whenever $Y_i = +1$ and $\vec{w} \cdot \vec{X}_i + b < 0$ whenever $Y_i = -1$. Equivalently, $Y_i(\vec{w} \cdot \vec{X}_i + b) > 0$ for all i . In this case, logit classification blows up in β , so alternative approaches must be employed.

For a given $\vec{w}$, the distance from a point $\vec{X}$ to the hyperplane is $|\vec{w} \cdot \vec{X} + b|/\|\vec{w}\|$. The *positive support vector* $\vec{X}^+(\vec{w})$ is the positive-class point closest to the hyperplane, and the *negative support vector* $\vec{X}^-(\vec{w})$ is the negative-class point closest to it. By rescaling $(\vec{w}, b)$ (which leaves the hyperplane unchanged), we may normalize so that

$$\vec{w} \cdot \vec{X}^+ + b = +1, \quad \vec{w} \cdot \vec{X}^- + b = -1. \quad (186)$$

Under this normalization, the *margin*, the distance between the two margin hyperplanes $\{\vec{w} \cdot \vec{x} + b = \pm 1\}$, equals

$$m(\vec{w}) = \frac{\vec{w} \cdot (\vec{X}^+ - \vec{X}^-)}{\|\vec{w}\|} = \frac{2}{\|\vec{w}\|}. \quad (187)$$

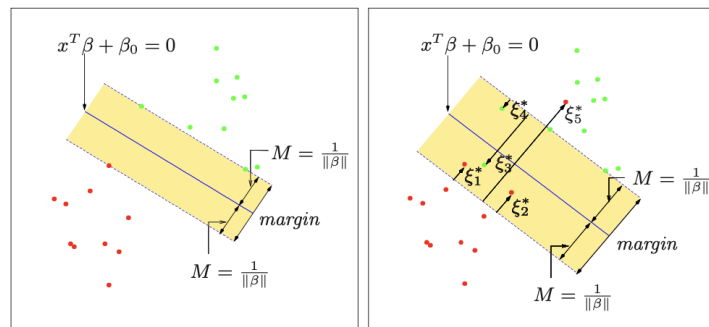


Figure 15: Hard- and soft-margin SVM

Theorem 9.1 (Hard-Margin SVM). *The hard-margin SVM maximizes the margin over all perfectly-separating hyperplanes, or equivalently solves the quadratic program*

$$\min_{\vec{w}, b} \frac{1}{2} \|\vec{w}\|^2 \quad \text{subject to} \quad Y_i(\vec{w} \cdot \vec{X}_i + b) \geq 1, \quad \forall i. \quad (188)$$

If a solution exists it is unique and is called the optimal separating hyperplane. The solution satisfies

$$\vec{w}_0 = \sum_{i=1}^n \alpha_i^0 Y_i \vec{X}_i, \quad (189)$$

where $\alpha_i^0 \geq 0$ are Lagrange multipliers that are nonzero only for the support vectors (observations on the margin boundary, where $Y_i(\vec{w}_0 \cdot \vec{X}_i + b_0) = 1$).

Proof. Introduce Lagrange multipliers $\alpha_i \geq 0$ for each constraint and form the Lagrangian

$$\mathcal{L}(\vec{w}, b, \vec{\alpha}) = \frac{1}{2} \vec{w} \cdot \vec{w} - \sum_{i=1}^n \alpha_i [Y_i(\vec{w} \cdot \vec{X}_i + b) - 1]. \quad (190)$$

The KKT stationarity conditions in $(\vec{w}, b)$ give

$$\frac{\partial \mathcal{L}}{\partial \vec{w}} = \vec{w} - \sum_i \alpha_i Y_i \vec{X}_i = 0, \quad \frac{\partial \mathcal{L}}{\partial b} = - \sum_i \alpha_i Y_i = 0, \quad (191)$$

so $\vec{w}_0 = \sum_i \alpha_i Y_i \vec{X}_i$ and $\sum_i \alpha_i Y_i = 0$. Substituting these back into $\mathcal{L}$ yields the dual objective

$$W(\vec{\alpha}) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j Y_i Y_j \vec{X}_i^T \vec{X}_j, \quad (192)$$

which is maximized over $\alpha_i \geq 0$, $\sum_i \alpha_i Y_i = 0$. By complementary slackness, $\alpha_i [Y_i(\vec{w}_0 \cdot \vec{X}_i + b_0) - 1] = 0$ for all i , so $\alpha_i > 0$ only for support vectors. The optimal classifier is

$$f(\vec{X}; \vec{w}_0, b_0) = \text{sgn} \left(\sum_{i=1}^n \alpha_i^0 Y_i \vec{X}_i^T \vec{X} + b_0 \right). \quad (193)$$

□

Remark 9.3 (Why Large Margins Generalize). The margin $2/\|\vec{w}_0\|$ measures how robustly the training data are separated: a large margin means that small perturbations of the data points do not alter the classification. This geometric stability translates directly into better out-of-sample prediction; the hard-margin SVM implicitly regularizes by constraining the norm $\|\vec{w}\|$, in the same spirit as Ridge regression.

When the data are not linearly separable, no feasible solution to the hard-margin problem exists. The *soft-margin SVM* relaxes the constraints by introducing slack variables.

Definition 9.7 (Soft-Margin SVM). Introduce nonnegative slack variables $\xi_i \geq 0$ that measure the degree of constraint violation, and solve

$$\min_{\vec{w}, b, \xi_i \geq 0} \frac{1}{2} \|\vec{w}\|^2 + C \sum_{i=1}^n \xi_i \quad \text{subject to} \quad Y_i(\vec{w} \cdot \vec{X}_i + b) \geq 1 - \xi_i, \quad \forall i. \quad (194)$$

The regularization parameter $C \in [0, \infty]$ controls the bias-variance tradeoff:

- $C \rightarrow 0$: all constraints are effectively dropped (any hyperplane is admissible);
- small C : wide soft margin, tolerating many misclassifications;
- large C : narrow soft margin, penalizing misclassification heavily;
- $C \rightarrow \infty$: recovers the hard-margin problem.

If a solution exists it is unique and is called the *optimal generalized hyperplane*.

Proposition 9.3 (Equivalence to Hinge Loss Minimization). *The soft-margin SVM is equivalent to the unconstrained minimization of the hinge loss plus an ℓ^2 regularization penalty. Setting $\lambda = 1/(2nC)$,*

$$\min_{(\vec{w}, b) \in \mathbb{R}^{d+1}} \frac{1}{n} \sum_{i=1}^n \ell(\vec{X}_i, Y_i; \vec{w}, b) + \lambda \|\vec{w}\|^2, \quad \ell(\vec{X}_i, Y_i; \vec{w}, b) := \max(0, 1 - Y_i(\vec{w} \cdot \vec{X}_i + b)). \quad (195)$$

The hinge loss is zero for correctly classified points lying beyond the margin and grows linearly with the distance of misclassified (or within-margin) points from the correct margin hyperplane.

Proof. Since the objective is increasing in ξ_i for $C \neq 0$, at optimality $\xi_i = \max(0, 1 - Y_i(\vec{w} \cdot \vec{X}_i + b))$. Substituting eliminates ξ_i from the problem, giving the unconstrained form. Dividing through by $2nC$ and setting $\lambda = 1/(2nC)$ yields the stated expression. $\square$

Definition 9.8 (Kernel Trick). The SVM extends to nonlinear classifiers via a two-step procedure:

1. Map $\vec{X} \in \mathbb{R}^d$ to an expanded feature space $\vec{Z} = \varphi(\vec{X}) \in \mathbb{R}^K$, $K \gg d$, using a fixed nonlinear transformation φ .
2. Find the optimal separating hyperplane in the feature space $\{(\varphi(\vec{X}_i), Y_i)\}_{i=1}^n$.

The key observation is that both the dual objective $W(\vec{\alpha})$ and the optimal classifier depend on inputs only through inner products $\langle \varphi(\vec{X}_i), \varphi(\vec{X}_j) \rangle$. A *kernel function* $K(\vec{u}, \vec{v}) = \langle \varphi(\vec{u}), \varphi(\vec{v}) \rangle$ computes this inner product in the feature space without explicitly constructing φ . The dual objective becomes

$$W(\vec{\alpha}) = \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j Y_i Y_j K(\vec{X}_i, \vec{X}_j), \quad (196)$$

and the classifier is $f(\vec{X}) = \text{sgn}\left(\sum_i \alpha_i^0 Y_i K(\vec{X}_i, \vec{X}) + b_0\right)$.

Theorem 9.2 (Mercer's Theorem). *A continuous symmetric function $K(\vec{u}, \vec{v})$ corresponds to an inner product $\langle \varphi(\vec{u}), \varphi(\vec{v}) \rangle$ in some feature space if and only if K is positive semi-definite: for all*

n , all $\vec{X}_1, \dots, \vec{X}_n$, and all $c_1, \dots, c_n \in \mathbb{R}$,

$$\sum_{i=1}^n \sum_{j=1}^n c_i c_j K(\vec{X}_i, \vec{X}_j) \geq 0. \quad (197)$$

This theorem justifies choosing a kernel directly: any positive semi-definite K implicitly defines a valid (possibly infinite-dimensional) feature map ϕ , so one need never construct ϕ explicitly.

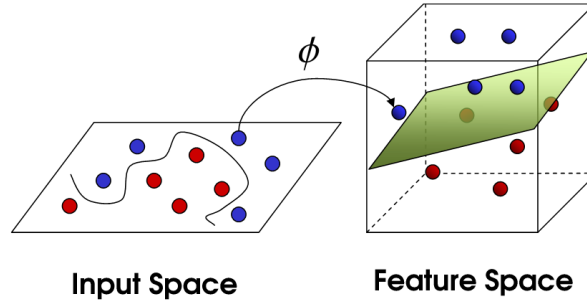


Figure 16: Data becomes linearly separable under the kernel mapping

Definition 9.9 (Common Kernel Functions). The most widely used kernels are:

- **Linear:** $K(\vec{u}, \vec{v}) = \vec{u} \cdot \vec{v}$. No transformation of features; equivalent to the original SVM. Works well in high-dimensional settings with low overfitting risk.
- **Polynomial of degree k :** $K(\vec{u}, \vec{v}) = (1 + \vec{u} \cdot \vec{v})^k$. Captures all monomial interactions up to degree k . For $p = 2, k = 2$, the induced feature map is $\phi(\vec{x}) = [1, \sqrt{2}x_1, \sqrt{2}x_2, \sqrt{2}x_1x_2, x_1^2, x_2^2]^T$.
- **Gaussian RBF:** $K(\vec{u}, \vec{v}) = \exp(-\gamma \|\vec{u} - \vec{v}\|_2^2)$. Highly flexible; each support vector generates a Gaussian bump centered at $\vec{X}_i$, and the decision boundary is a sum of localized effects. Overfits when γ is large (very narrow bumps). The induced feature space is infinite-dimensional.
- **Neural network:** $K(\vec{u}, \vec{v}) = \tanh(\gamma_1 \vec{u} \cdot \vec{v} + \gamma_2)$.

Kernel choice is fundamentally an empirical problem: it is equivalent to choosing a notion of similarity between observations. The recommended strategy is to start with a linear kernel and increase flexibility only when cross-validation confirms improvement.

Example 9.2 (Polynomial Kernel and Feature Map). Let $d = 2$ and take the degree-2 polynomial kernel $K(\vec{u}, \vec{v}) = (1 + \vec{u} \cdot \vec{v})^2$. Expanding:

$$(1 + u_1v_1 + u_2v_2)^2 = 1 + 2u_1v_1 + 2u_2v_2 + u_1^2v_1^2 + u_2^2v_2^2 + 2u_1v_1u_2v_2 \quad (198)$$

$$= \langle \phi(\vec{u}), \phi(\vec{v}) \rangle, \quad \phi(\vec{x}) = \left[1, \sqrt{2}x_1, \sqrt{2}x_2, \sqrt{2}x_1x_2, x_1^2, x_2^2 \right]^T. \quad (199)$$

The original $\mathbb{R}^2$ feature space is lifted to $\mathbb{R}^6$. Data that are not linearly separable in $\mathbb{R}^2$ (e.g., classes arranged in concentric rings) may be linearly separable after this transformation. Importantly, computing $K(\vec{u}, \vec{v}) = (1 + \vec{u} \cdot \vec{v})^2$ requires only a single dot product in $\mathbb{R}^2$, while explicitly working in $\mathbb{R}^6$ would be more expensive.

The SVM framework extends naturally to regression of a real-valued response Y via a modified loss function that is insensitive to small errors.

Definition 9.10 (ε -Insensitive Loss and SVR). The *support vector regression* (SVR) estimator minimizes

$$H(\vec{w}, b_0) = \sum_{i=1}^n V_\varepsilon(y_i - f(\vec{x}_i)) + \frac{\lambda}{2} \|\vec{w}\|^2, \quad f(\vec{x}) = \vec{w} \cdot \vec{x} + b_0, \quad (200)$$

where the ε -insensitive loss is

$$V_\varepsilon(r) = \begin{cases} 0 & \text{if } |r| < \varepsilon, \\ |r| - \varepsilon & \text{otherwise.} \end{cases} \quad (201)$$

The loss is zero for residuals smaller than ε (analogous to correctly classified points in the SVM) and grows linearly beyond that threshold (analogous to the hinge loss for soft-margin misclassifications). The ℓ^2 term $\frac{\lambda}{2} \|\vec{w}\|^2$ is a Ridge-type regularization.

Remark 9.4. The two parameters ε and λ play distinct roles. The parameter ε determines which observations are negligible in estimating $(\vec{w}, b_0)$: larger ε means fewer support vectors, since more residuals fall inside the insensitive band. Because V_ε is sensitive to the scale of y , practitioners typically normalize y to unit variance before fitting. The regularization parameter λ plays the usual bias-variance role and is selected by cross-validation.

Theorem 9.3 (SVR Solution and Sparsity). The minimizers $\hat{\vec{w}}$ and $\hat{b}_0$ of H admit the representation

$$\hat{\vec{w}} = \sum_{i=1}^n (\hat{\alpha}_i^* - \hat{\alpha}_i) \vec{x}_i, \quad \hat{f}(\vec{x}) = \sum_{i=1}^n (\hat{\alpha}_i^* - \hat{\alpha}_i) \langle \vec{x}, \vec{x}_i \rangle + \hat{b}_0, \quad (202)$$

where $\hat{\alpha}_i^*, \hat{\alpha}_i \geq 0$ solve the quadratic dual program

$$\min_{\alpha^*, \alpha} \varepsilon \sum_{i=1}^n (\alpha_i^* + \alpha_i) - \sum_{i=1}^n y_i (\alpha_i^* - \alpha_i) + \frac{1}{2} \sum_{i,i'=1}^n (\alpha_i^* - \alpha_i) (\alpha_{i'}^* - \alpha_{i'}) \langle \vec{x}_i, \vec{x}_{i'} \rangle, \quad (203)$$

subject to $0 \leq \alpha_i, \alpha_i^* \leq 1/\lambda$, $\sum_i (\alpha_i^* - \alpha_i) = 0$, $\alpha_i \alpha_i^* = 0$. The complementary slackness condition $\alpha_i \alpha_i^* = 0$ guarantees that only a sparse subset of observations have $\hat{\alpha}_i^* - \hat{\alpha}_i \neq 0$; these are the support vectors of the regression.

Remark 9.5 (SVR vs. Ridge in High Dimensions). When $d \gg n$, Ridge regression estimates a d -dimensional coefficient vector $\vec{\beta}$ by shrinking all components toward zero. SVR exchanges this for estimating at most n dual weights $(\hat{\alpha}_i^* - \hat{\alpha}_i)$ attached to the observed feature vectors $\{\vec{x}_i\}$: the effective dimension of the problem drops from d to (at most) n . SVR thus has a flavor of elastic net, but instead of eliminating individual covariates from $\vec{x}$ it eliminates entire observation vectors from the sum. This makes SVR particularly attractive in text and NLP applications where d (vocabulary size) can be orders of magnitude larger than n (number of time periods).

Definition 9.11 (Nonlinear SVR via Kernels). Since the SVR prediction $\hat{f}(\vec{x}) = \sum_i (\hat{\alpha}_i^* - \hat{\alpha}_i) \langle \vec{x}, \vec{x}_i \rangle + b_0$ and the dual objective both depend on inputs only through inner products $\langle \vec{x}_i, \vec{x}_j \rangle$, the kernel trick applies directly. Replacing the inner product by a kernel $K(\vec{x}, \vec{x}') = \langle \varphi(\vec{x}), \varphi(\vec{x}') \rangle$ yields the nonlinear SVR predictor

$$\hat{f}(\vec{x}) = \sum_{i=1}^n (\hat{\alpha}_i^* - \hat{\alpha}_i) K(\vec{x}, \vec{x}_i) + b_0. \quad (204)$$

Common choices are the polynomial kernel $(1 + \langle \vec{x}, \vec{x}' \rangle)^k$ and the Gaussian RBF $\exp(-\gamma \|\vec{x} - \vec{x}'\|^2)$.

Example 9.3 (Setup: Manela and Moreira (2017)). Manela and Moreira (2017) use SVR to construct a *news-implied volatility* index (NVIX) that extends the VIX back to 1890 using text from front-page Wall Street Journal articles. The goal is to measure the uncertainty concerns of the average investor over a century-long horizon and to identify which topics drive aggregate risk premia.

The regression model is

$$\text{VIX}_t - \overline{\text{VIX}} = w_0 + \vec{w}' \vec{x}_t + v_t, \quad t = 1, \dots, T, \quad (205)$$

where $\vec{x}_t \in \mathbb{R}^d$ is the vector of normalized n -gram (word and phrase) frequencies in month t , after eliminating words appearing fewer than three times. The dimension is $d = 468,091$ distinct n -grams, while the VIX training sample covers only $T = 276$ months (1986–2009). This is a canonical $d \gg T$ regime in which OLS is infeasible and Ridge would require storing and inverting a $468,091 \times 468,091$ matrix. SVR is the natural tool: it reduces the estimation problem from $O(d)$ to $O(T)$ by representing $\hat{\vec{w}}$ as a weighted average of support-vector word-usage profiles $\{\vec{x}_t\}_{t \in \mathcal{S}}$ for a sparse subset $\mathcal{S} \subset \{1, \dots, T\}$.

Remark 9.6 (SVM and SVR in Finance). The SVM and SVR inherit the regularization philosophy of Ridge and LASSO but express it geometrically: the margin constraint (SVM) or the ε -insensitive band (SVR) introduces sparsity in the dual rather than the primal, automatically selecting the most informative observations as support vectors. This dual sparsity is particularly valuable in high-dimensional text and alternative-data applications where $d \gg n$ is the rule rather than the exception. The kernel trick then enables nonlinear extensions at essentially no additional computational cost, since the dual problem always has dimension n regardless of the feature space dimensionality.

One limitation relative to LASSO or elastic net is that SVR does not perform variable (covariate) selection: the solution $\hat{\vec{w}} = \sum_t \hat{\alpha}_t \vec{x}_t$ uses all d covariates for each support vector $\vec{x}_t$, so inference on individual word coefficients is not directly available. Standard errors are also difficult to compute, as the dual representation does not map to a standard regression framework.

10 Neural Networks

Neural networks formalize and extend the sieve estimation framework of the previous section. A single hidden layer sigmoid neural network (sANN) was already introduced as a canonical nonlinear

sieve; here we develop the architecture, estimation, and regularization theory in detail, and connect the approach to the empirical asset-pricing and option-pricing literatures.

Definition 10.1 (Feed-Forward Neural Network). A *single hidden layer feed-forward neural network* with J_T hidden units approximates an unknown function $f : \mathbb{R}^p \rightarrow \mathbb{R}$ by

$$H(\vec{X}; \theta) = \beta_0 + \sum_{j=1}^{J_T} \beta_j S(\vec{\gamma}'_j \vec{X} + \gamma_{0,j}), \quad (206)$$

where:

- $\vec{X} \mapsto \vec{\gamma}'_j \vec{X} + \gamma_{0,j}$ is an affine transformation (linear combination plus shift) of the input vector;
- $S : \mathbb{R} \rightarrow \mathbb{R}$ is an *activation function*;
- $\theta := (\beta_0, \beta_1, \dots, \beta_{J_T}, \vec{\gamma}'_1, \dots, \vec{\gamma}'_{J_T}, \gamma_{0,1}, \dots, \gamma_{0,J_T})'$ is the full vector of parameters (called *weights*).

The scalars β_0 and $\gamma_{0,j}$ are called *biases*; $\vec{\gamma}_j$ controls the slope of S and $\gamma_{0,j}$ shifts the curve horizontally.

Remark 10.1 (Connection to Sieves). The sANN is precisely the nonlinear sieve: the basis functions $h_j(\vec{X}) = S(\vec{\gamma}_j \cdot \vec{X} + \gamma_{0,j})$ are parameterized by $(\vec{\gamma}_j, \gamma_{0,j})$, making the problem nonlinear. The *Universal Approximation Theorem* guarantees that, as $J_T \rightarrow \infty$, the class $\text{sANN}(J_T)$ is dense in the space of continuous functions (Cybenko, 1989; Hornik, Stinchcombe, and White, 1989).

The network can be represented in matrix form. Define augmented vectors $\tilde{\vec{X}}_t = (1, \vec{X}_t^T)^T$ and $\tilde{\vec{\gamma}}_j = (\gamma_{0,j}, \vec{\gamma}_j^T)^T$, and assemble them into matrices $X \in \mathbb{R}^{T \times (p+1)}$ and $\Gamma \in \mathbb{R}^{(p+1) \times J_T}$. Apply the activation function elementwise to form the *output matrix*

$$O(X\Gamma)_{[T \times (J_T+1)]} = \begin{pmatrix} 1 & S(\tilde{\vec{\gamma}}_1 \cdot \tilde{\vec{X}}_1) & \cdots & S(\tilde{\vec{\gamma}}_{J_T} \cdot \tilde{\vec{X}}_1) \\ \vdots & \vdots & \ddots & \vdots \\ 1 & S(\tilde{\vec{\gamma}}_1 \cdot \tilde{\vec{X}}_T) & \cdots & S(\tilde{\vec{\gamma}}_{J_T} \cdot \tilde{\vec{X}}_T) \end{pmatrix}. \quad (207)$$

Setting $\vec{\beta} = (\beta_0, \beta_1, \dots, \beta_{J_T})'$, the network output vector is compactly written as $H(X; \theta) = O(X\Gamma)\vec{\beta}$.

Definition 10.2 (Activation Functions). An *activation function* $S : \mathbb{R} \rightarrow \mathbb{R}$ introduces the non-linearity that makes neural networks universal approximators. A *squashing* (sigmoid) function satisfies $\lim_{u \rightarrow +\infty} S(u) = b$ and $\lim_{u \rightarrow -\infty} S(u) = a$ for finite $a < b$. Common choices include:

- **Logistic:** $S(x) = \frac{1}{1+e^{-x}}$, mapping $\mathbb{R} \rightarrow (0, 1)$.
- **Hyperbolic tangent:** $S(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$, mapping $\mathbb{R} \rightarrow (-1, 1)$.
- **Rectified Linear Unit (ReLU):** $S(x) = \max(0, x)$, a non-squashing function with no saturation for $x > 0$.
- **Radial Basis Function (RBF):** $S(x) = \exp(-x^2)$, a non-squashing function with localized response.

The logistic and hyperbolic tangent functions are bounded and smooth; ReLU is piecewise linear and is the dominant choice in modern deep networks because it avoids vanishing gradients and produces sparse activations.

Remark 10.2 (Geometric Interpretation). Each hidden unit j computes a scalar $\vec{\gamma}_j \cdot \vec{X}_t + \gamma_{0,j}$, whose level set $\{\vec{x} : \vec{\gamma}_j \cdot \vec{x} = \gamma_{0,j}\}$ is a hyperplane in $\mathbb{R}^p$ with normal $\vec{\gamma}_j$ and signed distance $\gamma_{0,j}/\|\vec{\gamma}_j\|$ from the origin. The activation function S maps this signed distance to a nonlinear response. With J hidden units, the covariate space is partitioned into at most $M(J, p)$ polyhedral regions, where $M(J, p) = M(J - 1, p) + M(J - 1, p - 1)$ with $M(1, p) = 2$ and $M(J, 1) = J + 1$. Each region receives a distinct affine prediction, so the network is equivalent to a piecewise-affine function.

Definition 10.3 (Empirical Risk Minimization for Neural Networks). Given a loss function L and sample $\{(Y_t, \vec{X}_t)\}_{t=1}^T$, the parameters are estimated by minimizing the empirical risk:

$$R_T(\theta) = \frac{1}{T} \sum_{t=1}^T L(Y_t, H(\vec{X}_t; \theta)). \quad (208)$$

For the quadratic loss, $R_T(\theta) = \frac{1}{T} \|\vec{Y} - H(\vec{X}; \theta)\|_2^2$. Because H is nonlinear in θ , no closed-form minimizer exists and the problem is non-convex, admitting multiple local minima.

Definition 10.4 (Gradient Descent). Starting from an initial guess $\theta^{(0)}$, *gradient descent* iterates

$$\theta^{(k+1)} = \theta^{(k)} - \eta_k \nabla_{\theta} R_T(\theta^{(k)}), \quad (209)$$

where $\eta_k > 0$ is the *learning rate*. Choosing η_k too large causes oscillation; too small causes slow convergence. *Stochastic gradient descent* (SGD) replaces the full gradient with a gradient computed on a random “mini-batch” of observations at each iteration, which reduces computational cost and introduces noise that can help escape shallow local minima.

The gradient $\nabla_{\theta} R_T$ is computed analytically via the *backpropagation* algorithm, which applies the chain rule along the computational graph $\vec{\gamma}_j \rightarrow z_{j,t} \rightarrow a_{j,t} \rightarrow H(\vec{X}_t; \theta) \rightarrow R_T(\theta)$, where $z_{j,t} = \vec{\gamma}_j \cdot \vec{X}_t + \gamma_{0,j}$ and $a_{j,t} = S(z_{j,t})$.

Proposition 10.1 (Backpropagation Gradients). *For the quadratic loss with a single hidden layer, define the output error $\delta_t = -\frac{2}{T}(Y_t - H(\vec{X}_t; \theta))$. Then*

$$\frac{\partial R_T}{\partial \beta_j} = \sum_{t=1}^T \delta_t S(z_{j,t}), \quad (210)$$

$$\frac{\partial R_T}{\partial \vec{\gamma}_j} = \sum_{t=1}^T \delta_t \beta_j S'(z_{j,t}) \vec{X}_t. \quad (211)$$

The error δ_t at the output layer is propagated backward through the hidden layer via the chain rule, weighted by $\beta_j S'(z_{j,t})$. This computation scales linearly in T and J_T , making training feasible for large networks.

Definition 10.5 (Regularization and Complexity Control). The number of hidden units J_T controls the bias-variance tradeoff of the network:

- If J_T is too small, the network underfits: high bias, low variance.
- If J_T is too large, the network may overfit: low training error, high out-of-sample error.

Overfitting is controlled via *weight decay* (an ℓ^2 penalty $\lambda\|\theta\|_2^2$ added to R_T) and *early stopping* (halting training when validation error begins to rise). Practical guidance is to start with a moderately large J_T and rely on regularization rather than deliberate underfitting; the network architecture determines *potential* complexity while regularization determines *effective* complexity.

Remark 10.3. The tuning parameter J_T may be selected by cross-validation over a grid (e.g., $J_T \in \{10, 20, 50, 100, 200\}$), choosing the value minimizing out-of-sample MSE. Information criteria are harder to apply here because the effective degrees of freedom of a nonlinear sieve are not easily characterized analytically.

A deep neural network (DNN) extends the single hidden layer architecture by stacking $L > 1$ hidden layers. While the universal approximation theorem holds for shallow networks, depth allows compositional functions to be approximated with exponentially fewer parameters than a shallow network of equivalent expressiveness (Mhaskar, Liao, and Poggio, 2017).

Definition 10.6 (Deep Neural Network). Let J_ℓ denote the number of hidden units in layer $\ell \in \{1, \dots, L\}$, and let $\Gamma_\ell \in \mathbb{R}^{(J_{\ell-1}+1) \times J_\ell}$ collect the weights for layer ℓ . Define the output matrix of layer ℓ recursively by

$$O_\ell(O_{\ell-1}(\cdot)\Gamma_\ell)_{[T \times (J_\ell+1)]} := \begin{pmatrix} 1 & S(\tilde{\gamma}_{1\ell} \cdot O_{1,\ell-1}) & \cdots & S(\tilde{\gamma}_{J_\ell\ell} \cdot O_{1,\ell-1}) \\ \vdots & \vdots & \ddots & \vdots \\ 1 & S(\tilde{\gamma}_{1\ell} \cdot O_{T,\ell-1}) & \cdots & S(\tilde{\gamma}_{J_\ell\ell} \cdot O_{T,\ell-1}) \end{pmatrix}, \quad (212)$$

with $O_0 = X$ (the raw input). The full network output is the composition

$$H(X; \theta) = O_L(\cdots O_2(O_1(X\Gamma_1)\Gamma_2) \cdots) \Gamma_L \vec{\beta}. \quad (213)$$

The total parameter count is $\sum_{\ell=1}^L (J_{\ell-1} + 1)J_\ell + J_L + 1$, which can easily exceed the sample size T , requiring strong regularization.

Definition 10.7 (Dropout). *Dropout* is a regularization technique for deep networks. At each training step, each neuron in each hidden layer is independently set to zero with probability $1 - q$ (typically $q = 0.5$). Formally, the layer- ℓ output under dropout is

$$y_i^{(\ell+1)} = \tilde{\gamma}_i^{(\ell+1)} \left(\vec{r}^{(\ell)} \odot \vec{z}^{(\ell)} \right) + b_i^{(\ell+1)}, \quad \vec{r}^{(\ell)} \sim \text{Bernoulli}(q)^{J_\ell}, \quad (214)$$

where $\odot$ denotes elementwise multiplication. At test time, the full network is used with weights scaled by q , recovering the same expected output as at training time. Dropout is equivalent to training an ensemble of $2^{\sum_{\ell} J_\ell}$ “thinned” sub-networks, which significantly reduces overfitting.

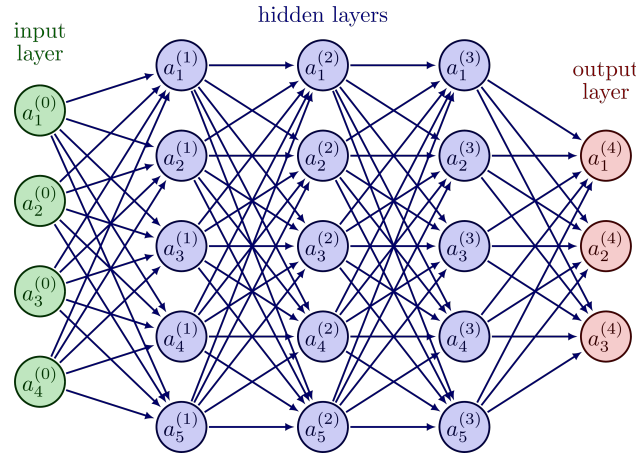


Figure 17: Neural network

Remark 10.4 (Challenges of Deep Network Training). Deep networks present two main challenges beyond those of shallow networks. First, the large parameter count demands correspondingly large training datasets and significant computational resources. Second, standard gradient descent suffers from *vanishing gradients*: when squashing activations (logistic, tanh) are saturated, $S'(z_{j,t}) \approx 0$, causing the backpropagated error signal to diminish exponentially with depth. ReLU activations largely resolve this by preserving gradient magnitude for positive pre-activations. Stochastic gradient descent with dropout and appropriate learning-rate schedules is the standard training recipe.

Architecture	Key advantage	Key challenge
Shallow (1 hidden layer)	Universal approximation; interpretable	May require very large J_T
Deep ($L > 1$ hidden layers)	Compositional efficiency; fewer units	Vanishing gradients; overfit risk
ReLU activation	No gradient saturation; fast training	Non-smooth; unbounded outputs
Dropout regularization	Ensemble effect; reduces overfit	Longer training; tuning q

Table 3: Comparison of neural network design choices.

The following results are drawn from Gu, Kelly, and Xiu (2020), who conduct a comprehensive horse race of machine learning methods for predicting the cross-section of stock returns.

Example 10.1 (Setup of Gu, Kelly, and Xiu (2020)). The model for stock i 's excess return is

$$r_{i,t+1} = g(\vec{z}_{i,t}) + \varepsilon_{i,t+1}, \quad \mathbb{E}_t[r_{i,t+1}] = g(\vec{z}_{i,t}), \quad (215)$$

where $\vec{z}_{i,t}$ is a vector of 94 firm characteristics (updated at monthly, quarterly, or annual frequency), 74 industry dummies, 8 macroeconomic predictors, and pairwise interactions between macroeconomic variables and firm characteristics. The dataset spans approximately 30,000 U.S. stocks from March 1957 to December 2016 (60 years). The sample is partitioned into an 18-year training window (1957–1974), a 12-year validation window (1975–1986), and a 30-year out-of-sample test

window (1987–2016). Methods compared include OLS, PLS, PCR, elastic net, generalized linear model with group lasso, random forest (RF), gradient boosted trees (GBRT), and neural networks with one to five hidden layers (NN1–NN5).

Example 10.2 (Setup: Correcting Option Pricing Models). Almeida, Fan, Freire, and Tang (2023) propose a two-step hybrid approach for predicting the implied volatility surface (IVS). The IVS is a bi-dimensional function $\sigma : \mathbb{R}^2 \rightarrow \mathbb{R}$ mapping moneyness $m = S/K$ and time to maturity τ to implied volatility. For a parametric model p (e.g., Black–Scholes, Heston) with model-implied IVS $\sigma^p(m, \tau)$, define the pricing-error surface

$$\varepsilon^p(m, \tau) = \sigma(m, \tau) - \sigma^p(m, \tau). \quad (216)$$

The procedure is:

1. Fit the parametric model to a cross-section of n option prices, obtaining residuals $\hat{\varepsilon}^p(m_{i,t}, \tau_{i,t}) = \sigma(m_{i,t}, \tau_{i,t}) - \hat{\sigma}^p(m_{i,t}, \tau_{i,t})$.
2. Fit a neural network $\hat{f}$ to the residuals by minimizing $\frac{1}{n} \sum_{i=1}^n [\hat{\varepsilon}^p(m_{i,t}, \tau_{i,t}) - f(m_{i,t}, \tau_{i,t})]^2$.
3. Predict as $\hat{\sigma}^p(m, \tau) + \hat{f}(m, \tau)$.

The intuition is that the neural network corrects the model’s systematic misspecification: the residual surface ε^p is smoother and simpler than the full IVS, and the Universal Approximation Theorem guarantees that a sufficiently wide or deep network can approximate it to arbitrary accuracy. Applying the neural network directly to the IVS (without a parametric template) is the special case where $\sigma^p = \text{const}$, corresponding to the Black–Scholes model.

Remark 10.5 (Neural Networks in Finance: Broader Perspective). The two empirical examples above illustrate a recurring theme: neural networks deliver the largest gains over parametric benchmarks when the true function has *unknown interaction structure* (return prediction, where characteristic interactions are complex and data-driven) or *known smooth structure with parametric misspecification* (option pricing, where the residual surface is smooth and well-approximated by a low-dimensional network). In both settings, the key design choice is whether to use the network as a direct universal approximator or as a residual corrector layered on top of a theoretically motivated parametric model. The latter approach is often preferable when interpretability or no-arbitrage constraints matter, as in derivatives pricing and risk management.

11 Large Language Models

Large language models (LLMs) are neural networks trained to model the probability distribution over sequences of text. They are the foundation of modern natural language processing and have demonstrated remarkable generalization across tasks ranging from translation and summarization to code generation and mathematical reasoning. LLMs are relevant both as a research topic and as a measurement tool in empirical finance: many economically relevant state variables (cash-flow news, managerial sentiment, regulatory uncertainty, discount-rate expectations) are observed only indirectly through language.

Definition 11.1 (Language Model). A language model defines a probability distribution over sequences of tokens $(t_1, t_2, \dots, t_T)$ drawn from a vocabulary $\mathcal{V}$. By the chain rule of probability:

$$\mathbb{P}(t_1, t_2, \dots, t_T) = \prod_{\tau=1}^T \mathbb{P}(t_\tau \mid t_1, \dots, t_{\tau-1}). \quad (217)$$

The fundamental task is *next-token prediction*: the model learns a sequence of conditional distributions, from which generation proceeds one token at a time by sampling.

Remark 11.1 (LLMs as Conditional Probability Models). An LLM is a very large conditional probability model. The object being estimated is a sequence of conditional distributions $\{P(\cdot \mid x_1, \dots, x_{t-1})\}_{t=1}^T$, making the framework naturally statistical. The model is generative because a joint distribution is built out of those conditionals via the chain rule. What is unusual relative to familiar econometric settings is the dimension and flexibility of the function class, not the logic of estimation.

Definition 11.2 (Tokenization). Raw text must first be converted into a finite alphabet of *tokens*. Tokens need not be whole words; typical tokens include words, subwords, punctuation, and special symbols. For example, “unbelievable” $\rightarrow$ (“un”, “believ”, “able”), which allows the alphabet to be smaller, since many words are constructed from these roots. The token vocabulary is finite, so tokenization maps arbitrary text into a manageable discrete alphabet, so that the input text can be given as a set of indices $(t_1, t_2, \dots)$ pointing to alphabet entries.

Token String	Token ID	Embedded Token Vector
'<s>' ->	0 ->	[0.1150, -0.1438, 0.0555, ...]
'<pad>' ->	1 ->	[0.1149, -0.1438, 0.0547, ...]
'</s>' ->	2 ->	[0.0010, -0.0922, 0.1025, ...]
'<unk>' ->	3 ->	[0.1149, -0.1439, 0.0548, ...]
'.' ->	4 ->	[-0.0651, -0.0622, -0.0002, ...]
' the' ->	5 ->	[-0.0340, 0.0068, -0.0844, ...]
',' ->	6 ->	[0.0483, -0.0214, -0.0927, ...]
' to' ->	7 ->	[-0.0439, 0.0201, 0.0189, ...]
' and' ->	8 ->	[0.0523, -0.0208, -0.0254, ...]
' of' ->	9 ->	[-0.0732, 0.0070, -0.0286, ...]
' a' ->	10 ->	[-0.0194, 0.0302, -0.0838, ...]
		...

Figure 18: The embedding matrix

Definition 11.3 (Token Embedding). Each token index $i \in \mathcal{V}$ is mapped to a learned continuous vector $\vec{e}(i) \in \mathbb{R}^d$ via an *embedding matrix* $E \in \mathbb{R}^{|\mathcal{V}| \times d}$, where the “one-hot” index vector picks a row. Similar contexts tend to push tokens toward similar numerical representations; this is the first stage of representation learning. Since the model must also track the position of each token in the sequence, a *positional encoding* $\vec{p}_\tau \in \mathbb{R}^d$ is added:

$$\vec{h}_\tau^{(0)} = E_{t_\tau} + \vec{p}_\tau. \quad (218)$$

The standard sinusoidal positional encoding uses $p_{\tau,2k} = \sin(\tau/10000^{2k/d})$ and $p_{\tau,2k+1} = \cos(\tau/10000^{2k/d})$, ensuring that the encoding varies smoothly with position. In modern models, learned position schemes are also common.

Remark 11.2 (Statistical Interpretation of Embeddings). Embeddings are a learned basis expansion: the model replaces a categorical variable by a dense, low-dimensional vector representation. This is analogous to latent-factor constructions such as PCA, but unlike PCA, the representation is learned jointly with the prediction objective rather than estimated from a covariance matrix. In finance language, embeddings can be viewed as learned latent characteristics.

Remark 11.3 (Necessity of Positional Information). Transformers process tokens largely in parallel, which is computationally attractive but removes any built-in notion of sequence order. Without position information, the model cannot distinguish “rates affect inflation” from “inflation affect rates.”

Definition 11.4 (Scaled Dot-Product Attention). Let $X \in \mathbb{R}^{T \times d}$ collect the current token representations of the input sequence in the embedding space. Self-attention forms three linear projections:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V, \quad (219)$$

where $W^Q, W^K \in \mathbb{R}^{d \times d_k}$ and $W^V \in \mathbb{R}^{d \times d_v}$ are learned projection matrices. The roles are:

- **Query:** what a token is looking for.
- **Key:** how a token advertises itself as relevant.
- **Value:** the information the token contributes.

The attention output is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V. \quad (220)$$

The (i, j) entry of $QK^T/\sqrt{d_k}$ is the attention score of position i attending to position j , measuring their compatibility. Softmax normalizes these scores into a probability distribution over positions with $\text{softmax}(\tilde{z})_i = \frac{e^{\tilde{z}_i}}{\sum_j e^{\tilde{z}_j}}$. Dividing by $\sqrt{d_k}$ stabilizes gradients by preventing the dot products from growing large in magnitude.

Remark 11.4 (One-Token Formula). For a single token i , the updated representation is thus

$$\tilde{v}_i = \sum_{j=1}^T w_{ij} v_j, \quad w_{ij} = \frac{\exp(q_i \cdot k_j / \sqrt{d_k})}{\sum_{\ell=1}^T \exp(q_i \cdot k_\ell / \sqrt{d_k})}. \quad (221)$$

The weights w_{ij} are endogenous and observation-specific: different tokens in the same sentence can use very different weighting schemes. Each token’s representation becomes a context-dependent weighted average of all other tokens’ value vectors.

Remark 11.5 (Attention as a Learned Kernel Smoother). Defining $W(X) = \text{softmax}(QK^T/\sqrt{d_k})$, attention can be written as $\text{Attention}(Q, K, V) = W(X)V$, where $W(X)$ is a data-dependent

weighting matrix with row i containing the weights used by token i to aggregate information from all other tokens. This is a learned, context-dependent smoothing operator that resembles a nonlinear kernel smoother, but the similarity metric is itself learned from data. Hence, attention is adaptive weighting under a learned geometry.

Example 11.1 (Attention as Soft Lookup). Consider the sentence: “The pizza came out of the oven and it tasted great.” For the token “it”, the query asks “what is my referent?” The key for “pizza” is a strong candidate, “oven” is weaker, and “tasted” is not a candidate. With $d_k = 2$, suppose the raw attention scores for “it” are $q_{it}K^T = [2.0, 1.0, 0.1]$. After softmax, the weights become approximately $(0.65, 0.24, 0.11)$: the representation of “it” shifts toward “pizza.” In another sentence, the same pronoun could attend to a different noun, since the representation is context-dependent.

Remark 11.6 (Causal Masking). In autoregressive language modeling, token t must not look ahead to future tokens. This is enforced by *causal masking*: setting the attention scores for $j > t$ to $-\infty$ before the softmax, so those positions receive zero attention weight.

Remark 11.7 (Self-Attention vs. Cross-Attention). In self-attention, queries, keys, and values all come from the same sequence X : the sequence updates itself using information from itself. In *cross-attention*, queries come from one sequence while keys and values come from another. Cross-attention is important in encoder–decoder architectures (e.g., machine translation), where the decoder attends to the encoder’s output.

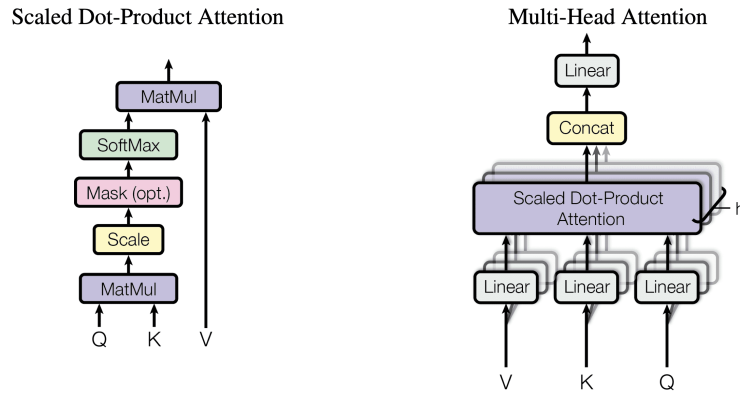


Figure 19: The attention mechanism

Definition 11.5 (Multi-Head Attention). Rather than computing a single attention function, *multi-head attention* runs H attention functions in parallel with different learned projections, then concatenates and projects the results:

$$\text{head}_h = \text{Attention}(XW_h^Q, XW_h^K, XW_h^V), \quad h = 1, \dots, H, \quad (222)$$

$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_H) W^O, \quad (223)$$

where $W_h^Q, W_h^K \in \mathbb{R}^{d \times d_k}$, $W_h^V \in \mathbb{R}^{d \times d_v}$, and $W^O \in \mathbb{R}^{Hd_v \times d}$ are learned projection matrices. Different heads can specialize: one head may focus on local syntax, another on long-range dependencies,

another on entity identification or causal relations. A single head is like one learned signal extractor; multi-head attention lets the model examine the same information set through several learned lenses at once.

Definition 11.6 (Transformer Block). A *transformer* is a stack of L identical blocks. Each block applies multi-head attention followed by a position-wise feedforward network, with residual connections and layer normalization at each step:

$$\vec{H}' = \text{LayerNorm}\left(\vec{H}^{(\ell-1)} + \text{MultiHead}(\vec{H}^{(\ell-1)})\right), \quad (224)$$

$$\vec{H}^{(\ell)} = \text{LayerNorm}\left(\vec{H}' + \text{FFN}(\vec{H}')\right), \quad (225)$$

where $\text{FFN}(\vec{h}) = W_2 \sigma(W_1 \vec{h} + \vec{b}_1) + \vec{b}_2$ is applied identically to each position, with $W_1 \in \mathbb{R}^{d \times d_{ff}}$ and $W_2 \in \mathbb{R}^{d_{ff} \times d}$.

Remark 11.8 (Residual Connections). The *residual connection* $\text{Output} = x + F(x)$ means the layer learns a correction to the input. If no change is needed, $F(x) \approx 0$. Residual connections preserve information from earlier layers, improve gradient flow in deep networks, and stabilize training. The analogy is: new estimate = old estimate + adjustment.

Remark 11.9 (Attention vs. Feedforward: Communication vs. Computation). Attention mixes information *across* tokens (inter-position communication), while the feedforward block transforms information *within* each token (intra-position computation). The feedforward network maps $\mathbb{R}^{d_{\text{model}}} \rightarrow \mathbb{R}^{d_{ff}} \rightarrow \mathbb{R}^{d_{\text{model}}}$, introducing nonlinearity through the activation function σ . Without the FFN, the model would be largely linear. Globally: attention = communication, FFN = computation.

Remark 11.10 (Layer Normalization). Layer normalization normalizes the activations across the feature dimension for each token independently: $\text{LayerNorm}(\vec{h}) = \gamma \odot \frac{\vec{h} - \mu}{\sigma} + \beta$, where μ and σ are the mean and standard deviation of $\vec{h}$, and γ, β are learned scale and shift parameters. This stabilizes training by preventing internal covariate shift, analogous to how batch normalization stabilizes activations across the batch dimension.

Definition 11.7 (Output Layer). After L transformer blocks, the final hidden state $H \in \mathbb{R}^{T \times d}$ is projected to the vocabulary via a linear map $W_{\text{vocab}} \in \mathbb{R}^{d \times |\mathcal{V}|}$:

$$Z = HW_{\text{vocab}} \in \mathbb{R}^{T \times |\mathcal{V}|}, \quad P(x_t | x_{<t}) = \text{softmax}(Z_t) \in \mathbb{R}^{|\mathcal{V}|}. \quad (226)$$

Each row of the output gives a probability distribution over the entire vocabulary for the next token at that position.

Remark 11.11 (Architectural Variants). Transformer architectures come in three forms:

- **Encoder:** processes the full input sequence bidirectionally (e.g., BERT). Useful for classification and embedding tasks.
- **Decoder:** generates text autoregressively with causal masking (e.g., GPT). This is the architecture underlying modern LLMs.

- **Encoder–Decoder:** maps input sequences to output sequences via cross-attention (e.g., the original Transformer for translation).

Modern LLMs are typically decoder-only models, trained autoregressively.

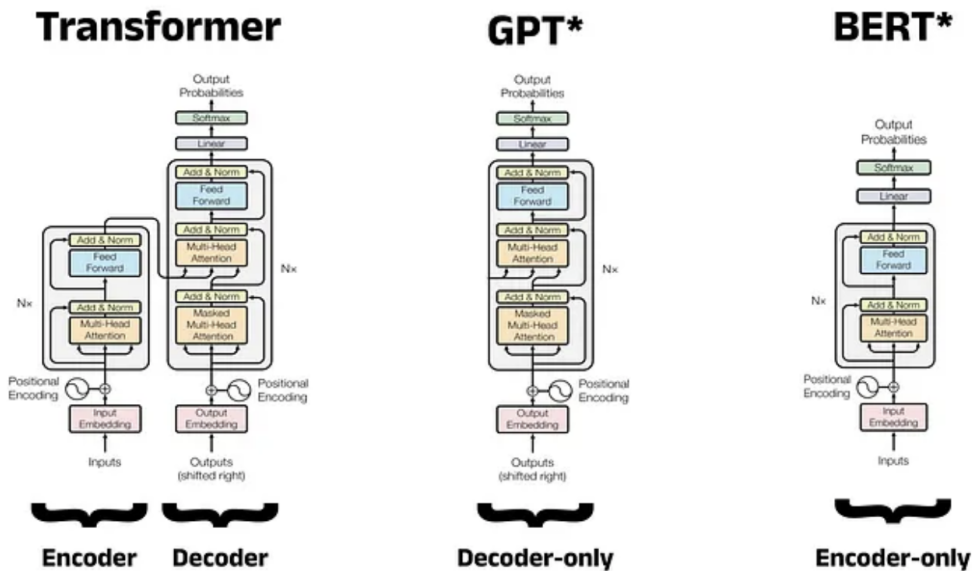


Figure 20: Various transformer architectures

Remark 11.12 (Transformers vs. Recurrent Neural Networks). In a recurrent neural network (RNN), information passes sequentially through hidden states, so learning long-range dependencies requires many steps and suffers from vanishing gradients. In a transformer, tokens interact directly via attention: any token can access any other in one step. This yields easier learning of long-range dependencies, more stable optimization, and highly parallelizable computation. The transformer has essentially replaced the RNN as the dominant sequence model.

Remark 11.13 (LLMs as Nonlinear Sieves). The transformer architecture is a direct generalization of the nonlinear sieve framework from Section 5. A single hidden layer sigmoid neural network (sANN) was introduced as a nonlinear sieve with parameterized basis functions $h_j(\vec{X}) = S(\vec{\gamma}_j \cdot \vec{X} + \gamma_{0,j})$. The transformer replaces the fixed nonlinearity S with the attention mechanism, a data-dependent, adaptive weighting operator, while retaining the same fundamental structure: a composition of learned nonlinear maps whose expressiveness grows with width (number of heads, hidden dimension) and depth (number of layers). The Universal Approximation Theorem guarantees that sufficiently large transformers can approximate any continuous function, just as for shallow networks, but depth allows compositional functions to be represented with exponentially fewer parameters.

Definition 11.8 (Cross-Entropy Loss). The model is trained to maximize the log-likelihood of

observed text, equivalently minimizing the *cross-entropy loss*:

$$\hat{\theta} = \arg \min_{\theta} \left[- \sum_{\tau=1}^T \log P_{\theta}(t_{\tau} \mid t_1, \dots, t_{\tau-1}) \right]. \quad (227)$$

This *next-token prediction* objective is self-supervised: the labels are derived from the data itself, requiring no human annotation. Training an LLM is maximum likelihood estimation.

Proposition 11.1 (Cross-Entropy and KL Divergence). *If $p^*(\cdot \mid x_{<t})$ denotes the true conditional distribution and $p_{\theta}(\cdot \mid x_{<t})$ the model, then the expected cross-entropy decomposes as*

$$\mathcal{H}(p^*, p_{\theta}) = \mathcal{H}(p^*) + D_{\text{KL}}(p^* \parallel p_{\theta}). \quad (228)$$

The entropy term $\mathcal{H}(p^)$ does not depend on θ . Minimizing expected cross-entropy is therefore equivalent to minimizing KL divergence from the true distribution.*

Proof. By definition,

$$\mathcal{H}(p^*, p_{\theta}) = - \sum_x p^*(x) \log p_{\theta}(x) = - \sum_x p^*(x) \log p^*(x) - \sum_x p^*(x) \log \frac{p_{\theta}(x)}{p^*(x)} \cdot (-1) \quad (229)$$

$$= \mathcal{H}(p^*) + \sum_x p^*(x) \log \frac{p^*(x)}{p_{\theta}(x)} = \mathcal{H}(p^*) + D_{\text{KL}}(p^* \parallel p_{\theta}). \quad (230)$$

□

Remark 11.14 (Empirical Scaling Laws). The scaling behavior of LLMs is empirically well-described by *power laws*. For a model with N parameters trained on D tokens using compute C , the test cross-entropy loss scales approximately as

$$\mathcal{L}(N, D, C) \approx aN^{-\alpha} + bD^{-\beta} + mC^{-\gamma} + c, \quad (231)$$

for constants $a, b, m, c, \alpha, \beta, \gamma > 0$. Performance improves smoothly and predictably with scale, but with diminishing returns. These scaling laws allow researchers to optimally allocate compute between model size, data, and training to minimize prediction error.

Remark 11.15 (Implications of Scaling). Several empirical findings are notable. First, larger models can be more sample-efficient, learning more per token of training data. Second, compute-efficient training often stops before full convergence. Third, the naive strategy of “train a smaller model much longer” is frequently suboptimal relative to training a larger model for fewer steps. Fourth, aspects of final model performance can be predicted from smaller pilot runs, giving firms a way to forecast whether a major training run is likely to justify its cost. The exact formula is model-dependent, but the broader lesson is robust: parameters, data, and compute all matter jointly, and none should be thought of in isolation.

Remark 11.16 (The Alignment Problem). Next-token prediction is a powerful objective, but it is not the same thing as optimizing for helpfulness, truthfulness, or safety. Internet text contains noise,

conflicts, and undesirable behavior; likelihood maximization does not directly encode preference for truthful or helpful outputs. A model can be statistically good at predicting internet text while still being poor at following a user’s intent. Post-training alignment addresses this gap through a three-stage pipeline: (1) supervised fine-tuning on curated responses, (2) reward model estimation from human preference rankings, and (3) reinforcement learning from human feedback.

Definition 11.9 (Pre-training and Fine-tuning). Modern LLMs are trained in two stages. In *pre-training*, the model is trained on a large corpus of unlabeled text using the next-token prediction objective, learning general linguistic structure, world knowledge, and reasoning patterns. In *fine-tuning* (also called supervised fine-tuning, SFT), the pre-trained model is further trained on a smaller dataset of curated prompt–response pairs, adapting its behavior to follow instructions. This *transfer learning* paradigm is highly parameter-efficient: fine-tuning updates far fewer parameters than training from scratch, and often achieves strong performance with limited task-specific data.

Definition 11.10 (Reward Modeling). For the same prompt, humans rank several candidate outputs. A separate *reward model* $R_\phi(x, y)$ is trained to predict those preferences, where x is a prompt and y is a response. The reward model is a learned proxy for the human preference relation: it maps (prompt, response) pairs to scalar scores such that $R_\phi(x, y_1) > R_\phi(x, y_2)$ whenever y_1 is preferred to y_2 .

Definition 11.11 (Reinforcement Learning from Human Feedback). Starting from the supervised model, *reinforcement learning from human feedback* (RLHF) optimizes the language model policy π_θ to maximize expected reward subject to a KL divergence penalty:

$$\max_{\theta} \mathbb{E}_{\vec{y} \sim \pi_\theta(\cdot|\vec{x})} [R_\phi(\vec{x}, \vec{y})] - \beta D_{\text{KL}}(\pi_\theta(\cdot|\vec{x}) \parallel \pi_{\text{ref}}(\cdot|\vec{x})), \quad (232)$$

where π_{ref} is the supervised fine-tuned model. The reward term encourages preferred outputs; the KL penalty prevents the policy from drifting too far from the reference, which could lead to reward hacking (exploiting weaknesses in the reward model to produce pathological but high-scoring outputs).

Remark 11.17 (Economic Interpretation of RLHF). The components of RLHF have natural economic analogues:

- Reward model $\approx$ learned utility function.
- Human rankings $\approx$ revealed preferences over outputs.
- KL regularization $\approx$ adjustment cost or penalty for departing from reference behavior, acting as a trust-region or model-risk control.
- RLHF $\approx$ constrained policy optimization under noisy preference data.

RLHF changes the behavior of the model but does not replace the knowledge acquired during pretraining. It adds a preference-alignment layer on top of the statistical model learned from data.

Remark 11.18 (Limitations). Even highly capable LLMs still display important limitations: hallucinations (plausible-sounding but factually incorrect outputs), brittleness outside the training distribution, sensitivity to prompt phrasing, and incomplete reliability in high-stakes settings. These limitations require careful human oversight in any production financial application.

Definition 11.12 (Econometric Mapping from Text to Returns). Let $\mathcal{T}_{i,t}$ denote the text associated with firm i at time t . An LLM produces an embedding

$$\vec{\phi}_{i,t} = \Phi(\mathcal{T}_{i,t}) \in \mathbb{R}^k, \quad (233)$$

where Φ is the LLM’s representation function. A predictive specification can then be written as

$$R_{i,t+1} = \alpha + \vec{\beta}' \vec{\phi}_{i,t} + \vec{\gamma}' \vec{Z}_{i,t} + \varepsilon_{i,t+1}, \quad (234)$$

where $\vec{Z}_{i,t}$ collects standard firm characteristics. The LLM is not the asset-pricing model itself; it is a *measurement device* that maps raw text into latent predictors. The downstream predictive regression remains an econometric object. More generally, a nonlinear factor model formulation is

$$R_{i,t+1} = f\left(\Phi(\mathcal{T}_{i,t}), \vec{Z}_{i,t}\right) + \varepsilon_{i,t+1}, \quad (235)$$

where the embedding vector plays the role of latent text factors and $f(\cdot)$ determines how those factors load into returns.

Remark 11.19 (What Latent Text Factors May Capture). The embedding $\vec{\phi}_{i,t}$ potentially summarizes information about cash-flow news, discount-rate news, managerial sentiment, uncertainty, disagreement, exposure to macro narratives, and risk related to regulation or technology. Unlike PCA-based factors, these representations are context-sensitive: the same word (“bank,” “volatility,” “exposure”) receives different embeddings depending on the surrounding text.

Remark 11.20 (Empirical Pipeline). A standard empirical pipeline for LLM-based asset pricing proceeds as follows: (1) collect text at date t ; (2) preserve timing to avoid look-ahead bias; (3) obtain embeddings or prompt-based labels; (4) combine with standard firm characteristics; (5) estimate predictive models for $R_{i,t+1}$; (6) evaluate out of sample and compute economic value. Good empirical design remains essential: one must respect the information set available at time t , benchmark against strong simpler models, use genuine out-of-sample validation, assess whether gains survive transaction costs, and distinguish statistical significance from economic significance.

Example 11.2 (Setup: Manela and Moreira (2017)). Manela and Moreira (2017) use SVR to construct a *news-implied volatility* index (NVIX) that extends the VIX back to 1890 using text from front-page Wall Street Journal articles. The regression model is

$$\text{VIX}_t - \overline{\text{VIX}} = w_0 + \vec{w} \cdot \vec{x}_t + v_t, \quad (236)$$

where $\vec{x}_t \in \mathbb{R}^d$ is the vector of normalized n -gram frequencies with $d = 468,091$ distinct n -grams and only $T = 276$ months of VIX training data. This canonical $d \gg T$ regime motivates the use of SVR and illustrates the broader opportunity: LLM-derived embeddings could replace the n -gram representation with a lower-dimensional, context-sensitive vector, potentially improving both statistical efficiency and economic interpretability.

Example 11.3 (Setup: Gu, Kelly, and Xiu (2020)). The comprehensive horse race of machine learning methods for stock return prediction by Gu, Kelly, and Xiu (2020) uses 94 firm characteristics,

74 industry dummies, and 8 macroeconomic predictors over 60 years of U.S. equity data. While the original study does not incorporate text, its framework — $r_{i,t+1} = g(\vec{z}_{i,t}) + \varepsilon_{i,t+1}$ — extends naturally to include LLM embeddings as additional entries in $\vec{z}_{i,t}$, providing a hybrid model that combines traditional characteristics with text-derived latent factors.

Example 11.4 (Setup: Correcting Option Pricing Models). Almeida, Fan, Freire, and Tang (2023) propose a two-step hybrid approach for predicting the implied volatility surface. For a parametric model p with model-implied IVS $\sigma^p(m, \tau)$, define the pricing-error surface $\varepsilon^p(m, \tau) = \sigma(m, \tau) - \sigma^p(m, \tau)$. A neural network $\hat{f}$ is fit to the residuals, and the prediction is $\hat{\sigma}^p(m, \tau) + \hat{f}(m, \tau)$. The neural network corrects the model's systematic misspecification: the residual surface is smoother and simpler than the full IVS, and the Universal Approximation Theorem guarantees that a sufficiently wide or deep network can approximate it to arbitrary accuracy.